



Comparison of a Classical RAG Pipeline and an Agent-Based Approach for Intelligent Knowledge Retrieval Across Enterprise Documentation Sources

Bachelorarbeit

submitted in partial fulfilment of the requirements for the degree of

Bachelor of Science

in the degree program New Study Informatik

at the Baden-Württemberg Cooperative State University Stuttgart

by

Liam Thomson

2026

Matrikulation Number, Course	3248571, DHBW-TINFO23
Training Company	Accenture, Kronberg im Taunus
Supervisors	Piotr Preuss
	Prof. Dr. Katja Wengler
Examiner	Dr. Prof. Kay Berkling

Confidentiality Notice

The present Bachelorarbeit titled *Comparison of a Classical RAG Pipeline and an Agent-Based Approach for Intelligent Knowledge Retrieval Across Enterprise Documentation Sources* is classified as confidential and is submitted solely for examination purposes in the New Study Informatik program at the Baden-Württemberg Cooperative State University Stuttgart. Any inspection or publication — including partial excerpts of the thesis — requires prior approval from Accenture.

Declaration

I hereby declare under oath:

1. that I have completed the present Bachelorarbeit on the topic *Comparison of a Classical RAG Pipeline and an Agent-Based Approach for Intelligent Knowledge Retrieval Across Enterprise Documentation Sources* independently and without external assistance;
2. that I have indicated the adoption of literal quotations from the literature and the use of the ideas of other authors at the corresponding points within the work;
3. that I have not submitted this Bachelorarbeit for any other examination.

I am aware that a false declaration will have legal consequences.

Baden-Württemberg Cooperative State University Stuttgart, 2026

Liam Thomson

Abstract

Enterprise knowledge is fragmented across structurally heterogeneous sources — issue trackers, collaborative wikis, and document repositories — each with different retrieval semantics. Classical Retrieval-Augmented Generation (RAG) pipelines apply uniform retrieval regardless of source type, leaving open the question of whether source-aware agent-based approaches offer a practical quality advantage.

This thesis implements and evaluates four system configurations over a shared corpus of 1,623 enterprise-proxy documents drawn from GitHub Issues, Markdown documentation, and CNCF PDF whitepapers. Ninety queries spanning factual, synthesis, ambiguous, and unanswerable types are evaluated across an original RAG pipeline, a single-agent ReAct system using LangGraph with three source-specific retrieval tools, a multi-agent coordinator applying fan-out retrieval followed by a single synthesis call, and an improved RAG pipeline incorporating Hypothetical Document Embeddings (HyDE), Maximal Marginal Relevance (MMR) reranking, and a synthesis-oriented generation prompt.

The single-agent system produced a 41% failure rate from hitting the iteration ceiling, 6.5× RAG latency, and an 11× token cost increase, with no quality advantage over the baseline on any measured metric. The multi-agent coordinator eliminated all failures, achieved the highest answer relevancy scores across both factual (0.818 vs. original RAG 0.792) and synthesis queries (0.770 vs. 0.679), and operated at 2.5× RAG cost. The improved RAG pipeline raised synthesis answer relevancy by nine percentage points at effectively identical cost, with a 1.7× latency increase attributable to the HyDE hypothesis generation call.

The improved RAG pipeline is the optimal upgrade path for deployments where cost and operational simplicity are primary constraints. The multi-agent coordinator is preferred when maximum answer quality across heterogeneous source types justifies a moderate cost premium. The single-agent ReAct pattern, as implemented, is unsuitable for production without structural fixes to its iteration-limit failure mode.

Table of Contents

1 Introduction **1**
- 1.1 The Enterprise Knowledge Problem . . . 1
- 1.2 Motivation for a Fresh Implementation . . . 2
- 1.3 Two Paradigms Under Comparison . . . 3
- 1.4 Research Questions . . . 4
- 1.5 Scope and Delimitations . . . 5

2 Theoretical Foundations **7**
- 2.1 Enterprise Knowledge Retrieval . . . 7
- 2.2 Retrieval-Augmented Generation . . . 9
- 2.3 Agent-Based Information Retrieval . . . 12
- 2.4 Evaluation Methodology . . . 16

3 System Design **21**
- 3.1 Shared Data Layer . . . 21
  - 3.1.1 Data Source Selection and Justification . . . 21
  - 3.1.2 Unified Document Schema . . . 23
  - 3.1.3 Ingestion Data Flow . . . 24
- 3.2 Classical RAG Pipeline Architecture . . . 25
  - 3.2.1 Pipeline Stages . . . 25
  - 3.2.2 Chunking Strategy . . . 26
  - 3.2.3 Embedding Model and Vector Store . . . 27
  - 3.2.4 Retrieval and Reranking . . . 28
- 3.3 Agent-Based System Architecture . . . 28
  - 3.3.1 The Agent Loop . . . 29
  - 3.3.2 Tools and Source Routing . . . 29
  - 3.3.3 Orchestration Logic . . . 30
  - 3.3.4 Multi-Agent Coordination Architecture . . . 31
- 3.4 Comparability Design Decisions . . . 32
  - 3.4.1 Shared Experimental Parameters . . . 32
  - 3.4.2 Defining Equivalent Effort . . . 33

4 Data and Evaluation Setup **35**
4.1 Dataset Construction . . . 35
4.1.1 Data Collection per Source Type . . . 35
4.1.2 Preprocessing and Normalisation . . . 36
4.1.3 Corpus Statistics . . . 37
4.2 Evaluation Query Set . . . 38
4.2.1 Synthetic QA Generation . . . 38
4.2.2 Query Taxonomy . . . 39
4.2.3 Manual Review and Filtering . . . 40
4.2.4 Final Query Set Statistics . . . 41
4.3 Evaluation Metrics Definition . . . 41
4.3.1 RAGAS Scores . . . 41
4.3.2 BERTScore and ROUGE-L . . . 42
4.3.3 Latency and Token Cost . . . 43
4.3.4 Agent-Specific Metrics . . . 44
4.3.5 Complexity and Maintainability Rubric . . . 45

5 Implementation **47**
5.1 Development Environment and Tooling . . . 47
5.2 RAG Pipeline Implementation . . . 49
5.2.1 Chunking and Embedding . . . 49
5.2.2 Retrieval and Reranking . . . 50
5.2.3 Failure Modes and Resolutions . . . 52
5.3 Agent Implementation . . . 53
5.3.1 Agent Loop . . . 53
5.3.2 Tool Implementations . . . 54
5.3.3 System Prompt and Routing Behaviour . . . 55
5.3.4 Runtime Failures and Resolutions . . . 56
5.3.5 Multi-Agent Coordination Architecture . . . 57
5.4 Logging and Instrumentation . . . 58

6 Evaluation Results **62**
6.1 Retrieval and Generation Quality . . . 62
6.2 Latency and Cost . . . 65
6.3 Implementation Complexity and Maintainability . . . 67
6.4 Robustness Observations . . . 69

7 Discussion **72**
7.1 Answering the Research Questions . . . 72
7.2 Trade-Off Analysis . . . 75
7.3 Limitations . . . 76

7.4 Connection to Prior Work 79

8 Conclusion **81**
8.1 Summary of Findings 81
8.2 Practical Recommendation 82
8.3 Future Work 83

Figures **i**

Tables **ii**

Listings **iv**

References **v**

1 Introduction

1.1 The Enterprise Knowledge Problem

Knowledge in large organisations is rarely located in one place. Relevant information is typically distributed across issue trackers, collaborative wikis, document repositories, and local file systems — each with its own search interface, data model, and access pattern. When a team member needs a precise, context-aware answer to an operational or client-related question, locating that answer requires either navigating multiple systems in sequence or relying on the tacit knowledge of a colleague who has already done so. This dependency on individual expertise creates fragility: when key contributors are unavailable, institutional knowledge becomes effectively inaccessible.

The problem is compounded by the structural diversity of enterprise sources. Issue tracker entries are short, structured, and dominated by metadata fields: status, assignee, priority, and linked items. Collaborative wiki pages are longer, discursive, and typically organised by project or function. Whitepapers and policy documents stored in SharePoint or similar repositories are formally structured but may span dozens of pages, with the most relevant passage buried in a single subsection. A retrieval system that performs well on one of these source types does not necessarily perform well on the others, because the retrieval semantics, document length distribution, and appropriate context window differ substantially between them.

The aggregate cost of this fragmentation is well recognised in practice. Time spent locating information rather than acting on it reduces operational efficiency; answers derived from incomplete retrieval produce decisions based on partial evidence; and the accumulation of tacit knowledge in individuals rather than in shared, queryable systems represents a structural risk for organisations with high staff turnover or geographically distributed teams. Automated knowledge retrieval systems, if they can operate reliably across heterogeneous sources, offer a direct remedy to each of these failure modes.

1.2 Motivation for a Fresh Implementation

Prior enterprise RAG deployments have addressed the knowledge retrieval problem by ingesting corporate knowledge objects, segmenting them into fixed sized text chunks, converting those chunks into dense vector embeddings, and storing them in a searchable index. At query time, a user's question is embedded and matched against the stored vectors; the most similar segments are passed to a large language model (LLM) as context, and a natural language answer is generated in response. This pattern is well established and has been applied in internal Accenture projects that provided the motivating context for this thesis.

Such deployments demonstrate that semantic retrieval over enterprise knowledge sources is technically feasible and useful in an operational context. Evaluation of these systems also surfaces a set of structural limitations that motivate the present thesis directly. Chunking strategy — the method by which documents are divided into retrievable segments — proved difficult to calibrate: segments that were too short lost the surrounding context necessary for coherent answers, while segments that were too long introduced irrelevant content into the retrieval signal. Source specific retrieval quality varied significantly, with issue tracker entries and wiki pages responding differently to the same embedding and retrieval configuration despite nominally belonging to the same index. The setup and maintenance burden was non-trivial, and extending the system to additional data sources required manual reconfiguration of the ingestion pipeline for each new source type.

These limitations are not specific to any single implementation; they are recognised failure modes of classical RAG pipelines applied to heterogeneous source collections (Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang, 2023). A fixed retrieval pipeline that applies the same chunking, embedding, and retrieval strategy uniformly across all source types may be structurally ill-suited to environments where source documents differ substantially in length, format, and information density. This structural mismatch is the central motivation for examining an agent-based alternative.

1.3 Two Paradigms Under Comparison

RAG addresses the knowledge limitation of LLMs by supplementing generation with externally retrieved content. In a classical RAG pipeline, a query is encoded as a vector, matched against a pre-built index of document embeddings, and the most relevant chunks are provided to the LLM as context for generating an answer. The approach grounds language model outputs in external knowledge, reducing factual hallucination and enabling knowledge updates without model retraining (Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, Küttler, Lewis, Yih, Rocktäschel, Riedel and Kiela, 2020). Classical RAG follows a fixed sequential structure: the retrieval step always occurs, always queries the same index, and always returns a fixed number of chunks, regardless of the query's nature, complexity, or appropriate source.

Agent-based systems represent a different paradigm. Rather than following a predetermined retrieval pipeline, an LLM-based agent reasons explicitly about what information is needed, selects the appropriate tool or data source to query, executes that query, observes the result, and iterates until the query is satisfactorily resolved (Yao, Zhao, Yu, Du, Shafran, Narasimhan and Cao, 2023). In a multi-source enterprise context, this means the agent can route a question about a software defect to an issue tracker retriever, a question about a deployment procedure to a documentation retriever, and a question about organisational standards to a PDF retriever — all within the same reasoning loop. Source routing, task decomposition, and iterative retrieval become explicit, observable steps in the agent's decision process rather than implicit assumptions baked into a fixed pipeline.

The two paradigms differ substantially in their design philosophy and expected performance characteristics. Classical RAG optimises for simplicity and predictability: a single retrieval step, a fixed context window, and a deterministic pipeline that is straightforward to deploy and monitor. Agent-based retrieval optimises for adaptability: dynamic source selection, multi-step reasoning, and the ability to refine a retrieval strategy based on intermediate observations. Whether this adaptability produces measurably better answers — and at what cost in latency, token usage, and implementation effort — is the central empirical question this thesis addresses.

1.4 Research Questions

Four research questions structure the comparative evaluation:

1. *How does a classical RAG pipeline compare to an agent-based approach in terms of latency, cost, and retrieval performance in enterprise environments?*

   This question establishes the baseline empirical comparison. Latency and cost are the primary operational constraints in enterprise deployment; retrieval performance — measured through context relevance, answer faithfulness, and answer relevance — determines whether either system is fit for the task at all. Without this foundation, comparisons of architectural sophistication are not grounded in observable outcomes.

2. *To what extent does an agent-based system improve source selection and answer quality compared to RAG?*

   Source selection accuracy — whether the system queries the appropriate document type for a given question — is a capability that agent-based systems are specifically designed to support but that classical RAG pipelines, by design, cannot implement. This question tests whether that architectural advantage translates into measurable quality gains, particularly on queries that span multiple source types or require iterative retrieval.

3. *What are the differences in implementation complexity and maintainability between both approaches?*

   A system that produces marginally better answers but requires substantially more effort to build, configure, and extend may not represent the right architectural choice for the majority of enterprise teams. By making implementation complexity an explicit evaluation dimension — assessed through code volume, configuration depth, and the effort required to onboard a new data source — this question ensures that the comparison is practically actionable.

4. *Under which conditions is each approach preferable in practice?*

   The first three questions yield comparative measurements across individual dimensions; the fourth synthesises them into a conditional recommendation. Given a specific combination of corpus characteristics, query complexity distribution,

team expertise, and resource constraints, which architecture is the more appropriate choice? This question is the applied output of the thesis and its primary contribution to practitioners facing this architectural decision.

1.5 Scope and Delimitations

This thesis focuses on the comparative evaluation of two retrieval architectures — a classical RAG pipeline and a ReAct-based agent — applied to a shared corpus of enterprise-proxy documents. Both systems are implemented as functional research prototypes and evaluated under identical conditions; the findings are intended to inform architectural decisions, not to constitute a production deployment recommendation.

Several topics fall outside the scope of this work. Production hardening — including security, access control, rate limiting, and audit logging — is not addressed; both systems are evaluated in a controlled, single-user experimental environment. User studies and human evaluation of answer quality are excluded; evaluation relies on automated metrics supplemented by LLM-as-judge scoring, a methodology validated by Es, James, Anke and Schockaert (2023) as sufficiently correlated with human judgements for comparative evaluation purposes. The primary comparison is between a classical RAG pipeline and a single-agent ReAct system — two architectures planned from the outset and evaluated under identical conditions. Two extensions arose during the evaluation: a multi-agent coordinator, designed in response to structural failure modes identified during single-agent execution, and an improved RAG configuration applying HyDE, MMR, and a revised synthesis prompt to the baseline pipeline. Both extensions are reported in the evaluation alongside the primary comparison, representing methodological progression from observation of failure through diagnosis to corrective architectural design. Fine-tuning of either the LLM or the embedding model on enterprise-specific data is not performed; both systems rely on general-purpose pre-trained models.

The document corpus consists of three open, publicly available source types: GitHub Issues, Markdown documentation, and PDF whitepapers. These are selected as structural proxies for the enterprise source types — Jira tickets, Confluence wiki pages, and SharePoint documents — that characterise the enterprise knowledge retrieval problem.

This proxy design enables reproducible academic evaluation without requiring proprietary data access. It introduces a generalisability limitation, which is addressed in Chapter 7.

2 Theoretical Foundations

2.1 Enterprise Knowledge Retrieval

Enterprise knowledge retrieval encompasses the challenge of surfacing relevant, context-aware information from collections of heterogeneous documents distributed across organisational systems. Modern enterprises accumulate knowledge across structurally distinct source types—issue trackers, collaborative wikis, and document repositories—each of which imposes different constraints on how information is stored, queried, and consumed. Document length, content vocabulary, and query structure each differ systematically across source types, and a retrieval configuration calibrated for one will perform poorly on at least one of the others.

The three source types employed in this thesis illustrate the range of structural variation encountered in practice. Issue tracker entries, represented here by GitHub Issues as a proxy for Jira tickets, are short and metadata-rich, with discrete fields for status, labels, and assignees alongside free-text comment threads. Collaborative wiki pages, represented by Markdown documentation as a proxy for Confluence, are discursive and medium-length, organised by human-authored headings into flowing prose. Formal document repositories, represented by PDF whitepapers as a proxy for SharePoint, contain long, hierarchically structured texts with dense technical content and standardised sections. A retrieval approach that treats these source types uniformly risks systematic degradation on at least one category, as enterprise RAG deployments consistently demonstrate when a uniform retrieval configuration produces uneven results across structurally distinct document classes (Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang, 2023).

Latency expectations compound the difficulty of the retrieval problem. Enterprise users tolerate response times that vary with query complexity: simple factual lookups carry

an implicit expectation of near-instantaneous response, whereas cross-document synthesis queries, which require combining evidence from multiple sources, are accepted with longer wait times. A retrieval system must therefore balance thoroughness against speed in a manner that is sensitive to query type, not only to document volume.

Traditional enterprise search has long relied on lexical matching methods, of which the BM25 ranking function is the canonical representative. BM25 scores documents by term frequency and inverse document frequency, yielding high precision for queries that share surface tokens with the target document. Its limitations emerge when the query and the relevant passage share meaning without sharing vocabulary: a query for "deployment instructions" will fail to retrieve a document whose relevant passage reads "steps to install and configure the service," because no term overlap exists. This vocabulary mismatch problem is a persistent source of recall loss in operational retrieval systems.

Federated search addresses a different dimension of the enterprise retrieval problem by coordinating queries across multiple disparate systems and aggregating the results into a unified response. It solves the organisational challenge of querying siloed repositories without requiring data migration, but it does not address the semantic gap: each constituent system still applies lexical matching internally, and the aggregation layer has no mechanism for resolving conceptual relationships across sources. Federated search thus increases coverage without improving the quality of individual matches, leaving cross-source synthesis queries — those that require reconciling information distributed across an issue thread, a wiki page, and a whitepaper — beyond its reach.

The limitations of both approaches converge on a shared root cause: neither represents documents or queries as continuous semantic objects. Keyword search operates on discrete tokens; federated search multiplies the number of token-matching engines without changing the underlying representation. Semantic retrieval methods, which map documents and queries into dense vector representations where proximity reflects meaning rather than surface form, address this root cause directly. The two paradigms examined in this thesis — classical RAG and agent-based retrieval — both build on semantic representations as their foundation, and the comparative evaluation consequently begins from a baseline that lexical search cannot achieve.

2.2 Retrieval-Augmented Generation

RAG extends the capabilities of LLMs by grounding generation in dynamically retrieved external knowledge, rather than relying solely on parameters encoded during training (Lewis et al., 2020). The original formulation by Lewis et al. (2020) decomposes the system into two learnable components: a retriever, parameterised as a Dense Passage Retriever (DPR) that estimates the probability of a document given a query, and a generator, parameterised as a sequence-to-sequence model that conditions its output on both the query and the retrieved passages. A critical architectural property of this design is that the non-parametric retrieval index can be updated or expanded without retraining the generator, making the system's knowledge base directly inspectable and modifiable (Lewis et al., 2020).

The core RAG pipeline proceeds across three sequential phases: indexing, retrieval, and generation. During indexing, source documents are segmented into chunks, encoded by an embedding model into dense vector representations, and stored in a vector index alongside their source metadata. At query time, the retrieval phase encodes the user query using the same embedding model, computes cosine similarity against the stored vectors and returns the top-k most relevant chunks. These retrieved passages are then concatenated with the original query and passed to the generator as a prompt context, from which the final response is produced. The generator therefore operates on a composed input that merges parametric knowledge with non-parametric evidence, reducing the risk of hallucination on knowledge-intensive tasks.

The RAG landscape has since diversified substantially, and Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang (2023) organise this development into three paradigms: Naive RAG, Advanced RAG, and Modular RAG. Naive RAG implements the core pipeline described above without modification — a single retrieval step, no query transformation, and no post-retrieval filtering — and serves as the tractable baseline against which improvements are measured. However, Naive RAG exhibits well-documented failure modes: retrieval precision is often low, with irrelevant chunks included in the context, while recall gaps mean that relevant passages may not surface at all (Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang, 2023). These deficiencies propagate directly into generation, where the presence of contradictory or off-topic retrieved content can cause the model to produce responses that conflict with the retrieved evidence.

Advanced RAG addresses these failure modes by introducing pre-retrieval and post-retrieval processing stages around the core pipeline. The most impactful pre-retrieval technique is Hypothetical Document Embedding (HyDE), which addresses a fundamental asymmetry in dense retrieval: query vectors and document vectors occupy systematically different regions of the same embedding space, because a question and the passage that answers it differ in linguistic register even when they share meaning (Gao, Ma, Lin and Callan, 2023). HyDE resolves this mismatch by prompting the language model to generate a short hypothetical passage that directly answers the query, then embedding that passage—rather than the literal query—as the search vector. Because the generated hypothesis is phrased in the linguistic style of a document answer rather than a question, it aligns naturally with the embedding space of stored chunks, substantially improving recall on queries that are short, ambiguous, or structurally distant from the relevant indexed content (Gao, Ma, Lin and Callan, 2023). The original query is preserved for cross-encoder reranking, ensuring that final relevance scoring remains grounded in the actual question regardless of how the hypothesis was phrased. Ferrazzi, Cvjeticanin, Piraccini and Giannuzzi (2026) confirm empirically that HyDE query rewriting yields the strongest retrieval quality gains among standalone pre-retrieval techniques, with the largest improvements occurring on datasets where user queries diverge structurally from the target documents.

Post-retrieval processing refines the candidate set before it reaches the generator. Cross-encoder reranking scores each retrieved passage against the query jointly rather than independently, producing substantially better relevance ordering than the inner-product score from the initial vector search (Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang, 2023). Relevance-only ranking, however, can populate the top-k set with near-duplicate passages from a single document section, supplying the generator with redundant evidence while excluding complementary material from other documents. Maximal Marginal Relevance (MMR) (Carbonell and Goldstein, 1998) addresses this by iteratively selecting the passage that maximises a linear combination of relevance and dissimilarity to already-selected passages:

$$\text{MMR} = \arg\max_{d_i \in R \setminus S} \left[\lambda \cdot \text{Rel}(d_i) - (1 - \lambda) \cdot \max_{d_j \in S} \text{Sim}(d_i, d_j) \right] \tag{2.1}$$

where R is the ranked candidate set, S is the set of already-selected passages, $\lambda \in [0, 1]$ controls the trade-off between relevance and diversity, and $\text{Sim}(\cdot, \cdot)$ is cosine similarity of normalised embedding vectors. At $\lambda = 1$ the criterion degenerates to pure relevance

ranking; lower values shift selection towards passages that are topically distinct from those already chosen. Context compression further reduces noise by truncating or filtering the selected passages to the most answer-relevant spans before they reach the generator (Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang, 2023). The improved RAG pipeline evaluated in this thesis applies all four of these techniques—HyDE pre-retrieval, cross-encoder reranking, MMR diversification, and a structured synthesis prompt—and is therefore positioned as an Advanced RAG system; a baseline configuration applying only cross-encoder reranking, without HyDE, MMR, or a structured synthesis prompt, is evaluated in parallel to isolate the contribution of each enhancement. Both variants apply a single fixed retrieval step without source routing or adaptive retrieval decisions.

Modular RAG deconstructs the pipeline further into interchangeable components and introduces new capabilities not present in either earlier paradigm. Routing directs different query types to different retrieval strategies or data sources; scheduling determines whether retrieval is needed at all for a given query; and adaptive retrieval enables the system to retrieve multiple times, conditioning each subsequent query on prior observations (Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang, 2023). These capabilities bring Modular RAG into conceptual proximity with agent-based systems, in which source routing and adaptive retrieval are externalised as explicit reasoning steps rather than embedded as pipeline parameters.

The effectiveness of a RAG system depends considerably on chunking strategy, which determines the granularity and coherence of the indexed units. Fixed-size chunking is computationally cheap but routinely splits passages at semantically arbitrary boundaries, degrading retrieval precision. Sentence-level chunking respects natural language units yet produces short chunks that may lack sufficient context for accurate relevance scoring. The Small2Big — or hierarchical — strategy offers the strongest balance: small chunks are indexed and retrieved for retrieval precision, but the generator receives the surrounding parent chunk as context, preserving local coherence (Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang, 2023). Sliding window chunking with configurable overlap reduces the risk of splitting relevant spans across adjacent chunks and remains a practical complement to sentence-level segmentation.

Embedding model choice determines the representational capacity of the retrieval index. Sparse retrieval methods, such as BM25, operate on exact lexical overlap and serve as strong baselines for short, precise queries, but generalise poorly to paraphrase

or synonym variation. Dense retrieval models such as the BGE and E5 families learn semantic representations that capture meaning beyond surface form, enabling retrieval across queries that share intent but not vocabulary (Lewis et al., 2020). In practice, hybrid retrieval combining sparse and dense signals can outperform either method alone on heterogeneous corpora.

Despite these enhancements, Advanced RAG remains a static pipeline: retrieval is always performed, always from a single index, and always at a fixed point in the generation process. This design assumption becomes a liability when the information need is ambiguous, when multiple retrieval steps are required to integrate evidence across documents, or when retrieval is unnecessary and introduces noise. Asai, Wu, Wang, Sil and Hajishirzi (2023) demonstrate that indiscriminate retrieval, regardless of whether the query requires it, reduces model performance; removing adaptive retrieval decisions from their system causes accuracy to fall from 54.9 to 42.6 on PopQA. Jiang, Xu, Gao, Sun, Liu, Dwivedi-Yu, Yang, Callan and Neubig (2023) show that a single retrieval step is insufficient for long-form generation tasks where information needs evolve across the output, and that active, forward-looking retrieval outperforms a fixed single-step baseline by a substantial margin on multi-hop reasoning tasks. Taken together, these findings suggest that the architectural ceiling of static retrieval pipelines motivates a fundamentally different approach — one in which retrieval decisions are made dynamically by a system capable of selecting sources, composing multi-step queries, and adjusting its strategy based on intermediate results.

2.3 Agent-Based Information Retrieval

Agent-based systems extend language model capabilities by embedding the model within a decision-making cycle that interleaves reasoning, action, and environmental feedback. Where a classical RAG pipeline executes a fixed sequence — retrieve once, then generate — an agent-based system treats retrieval as one of many possible actions, selected dynamically based on the current query state and prior observations. Xi, Chen, Guo, He, Ding, Hong, Zhang, Wang, Jin, Zhou, Zheng, Fan, Wang, Xiong, Liu, Zhou, Wang, Jiang, Zou, Liu, Yin, Dou, Weng, Cheng, Zhang, Qin, Zheng, Qiu, Huang and Gui (2023) define this class of system through four distinguishing properties: autonomy, in that the agent operates without direct human intervention at each step; reactivity, in that it responds to observations returned by tools; pro-activeness, in that it pursues

a goal across multiple steps; and social ability, in that it can in principle coordinate with other agents or human users. These properties collectively describe a system that decides *how* to retrieve information, rather than one that simply retrieves it.

The ReAct framework, introduced by Yao et al. (2023), provides the theoretical backbone for this class of agent by interleaving verbal reasoning traces with discrete tool actions in a repeating loop. Each cycle consists of three elements: a Thought, which is a free-form reasoning step in which the model assesses its current state and decides on a next action; an Action, which is a structured call to a defined tool with specific arguments; and an Observation, which is the value returned by that tool and appended to the context before the next Thought step. The agent continues cycling until it determines that sufficient information has been gathered to produce a final answer. Yao et al. (2023) demonstrate that this interleaving significantly outperforms either reasoning or acting in isolation, reporting 27.4 exact-match accuracy on HotpotQA and 71% task success on ALFWorld. Critically, the Thought traces are human-readable, making the agent's decision process auditable in a way that the single-pass RAG pipeline is not.

Tool use is the mechanism through which the agent grounds its reasoning in external information sources, and it directly addresses several limitations of plain language model inference. Schick, Dwivedi-Yu, Dessì, Raileanu, Lomeli, Zettlemoyer, Cancedda and Scialom (2023) identify four such limitations: the model's knowledge is bounded by its training cut-off; in-context missteering can cause factually incorrect completions; hallucination degrades reliability on knowledge-intensive tasks; and domain-specific sub-tasks such as arithmetic require capabilities the base model lacks. The filtering criterion that distinguishes a useful tool call from a redundant one is whether the tool's response reduces future generation loss by at least a fixed threshold — a principle that generalises beyond Toolformer's specific implementation to any agent that must decide whether to query an external source (Schick et al., 2023). The ReAct loop overcomes the sequential dependency limitation inherent in single-step tool use by making each Observation available as context for the subsequent Thought, enabling the agent to compose tool calls iteratively across a multi-hop query.

Wang, Ma, Feng, Zhang, Yang, Zhang, Chen, Tang, Chen, Lin, Zhao, Wei and Wen (2023) decompose agent architecture into four interacting modules that map directly onto the system built in this thesis. The profiling module defines the agent's role and constraints, corresponding to the system prompt that instructs the agent to function as an enterprise retrieval assistant with access to source-specific tools. The memory

module encompasses both short-term working memory — the in-context accumulation of the running ReAct trace — and long-term semantic memory, implemented here as the per-source document indexes. The planning module governs how the agent sequences its actions; Wang et al. (2023) distinguish single-path planning (Chain-of-Thought), multi-path planning (Tree-of-Thought), and feedback-driven planning (ReAct), with ReAct identified as optimal for multi-hop retrieval tasks where each step must inform the next. The action module defines the typed tool calls available to the agent, corresponding in this thesis to three source-specific retrieval tools: one for GitHub Issues, one for Markdown documentation, and one for PDF whitepapers.

The Cognitive Architectures for Language Agents (CoALA) framework from Sumers, Yao, Narasimhan and Griffiths (2023) provides a formal basis for distinguishing agent-based systems from classical RAG. CoALA partitions agent memory into working memory (the current query, retrieved passages, and running trace), episodic memory (past trajectories), semantic memory (long-term unstructured knowledge), and procedural memory (implicit weights plus explicit prompt templates and tool definitions). Under this taxonomy, classical RAG is a constrained agent that reads from a fixed, read-only semantic memory with no surrounding decision cycle — Sumers et al. (2023) describe retrieval-augmented methods as "retrieving from a semantic memory of unstructured text" and position classical RAG as the degenerate case in which the agent has no planning stage and no adaptive retrieval decision. The thesis agent adds a decision cycle around retrieval, enabling conditional and iterative access to multiple semantic memory stores rather than a single unified index.

Shinn, Cassano, Berman, Gopinath, Narasimhan and Yao (2023) extend the ReAct loop with verbal self-reflection, in which a dedicated self-reflection model generates a textual summary of why a prior attempt failed, storing that summary in episodic memory for the next attempt. The resulting system achieves 97% task success on ALFWorld, compared to 78% for standard ReAct, demonstrating that verbal reinforcement of the decision cycle substantially improves agent performance on complex sequential tasks. This capability is not implemented in the agent evaluated in this thesis, as queries are treated as independent and no learning is persisted between runs; Reflexion nonetheless represents a natural extension direction that would bring the system closer to a continuously adaptive enterprise retrieval agent.

A structurally distinct extension of the single-agent model is the multi-agent coordination architecture, in which a coordinator node dispatches sub-agents to distinct

retrieval sources and a dedicated synthesiser node combines all retrieved evidence in a single language model call. Each sub-agent is responsible solely for querying its assigned source and returning a ranked passage set; no language model call occurs during retrieval, and the synthesiser receives the pooled evidence as a single bounded context. This decoupling of retrieval from reasoning eliminates two failure modes that sequential ReAct loops exhibit under constrained token budgets: context saturation, in which accumulated observations across successive tool calls crowd out the token budget for the final answer; and cascading iteration limit violations, in which exhausting the available steps while searching one source leaves subsequent sources unqueried. Xi et al. (2023) identify parallel sub-agent dispatch as the primary scalability mechanism for settings in which sub-goals are structurally independent — a condition satisfied in enterprise retrieval when each source type is semantically distinct and queryable without reference to the others. (2025) demonstrate that the fundamental limitation of static RAG pipelines is their constraint on what evidence the language model can access, and that overcoming this constraint through structured, model-driven retrieval over enterprise corpora yields a 5.9× improvement in recall relative to single-pass retrieval baselines. The multi-agent architecture evaluated in this thesis addresses the same constraint by always querying all three source types per query, ensuring that the synthesiser receives evidence from every domain in a bounded, fixed-size context regardless of query characteristics.

The contrast between classical RAG and agent-based retrieval is most sharply visible across the evaluation dimensions examined in this thesis. Classical RAG routes all queries through a single unified index, retrieves once per query, and produces a response in a single language model call, resulting in lower latency and lower token expenditure. The agent-based system routes each query to one or more source-specific tools, retrieves conditionally and iteratively based on accumulated observations, and may invoke the language model several times before producing a final response. This iterative structure improves support for multi-hop queries that require synthesising information from multiple documents or sources, but imposes measurable costs in latency and token consumption — a trade-off Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang (2023) identify as central to the design space of advanced retrieval systems. The readable Thought traces produced by the ReAct loop partially offset the higher cost by providing interpretability that the black-box RAG pipeline cannot match.

2.4 Evaluation Methodology

Evaluating retrieval-augmented systems presents a challenge that traditional natural language processing (NLP) benchmarks are not designed to address: the quality of a generated answer depends jointly on retrieval precision and generation faithfulness, and these two failure modes require distinct measurement instruments. The evaluation framework employed in this thesis combines three complementary layers—reference-free retrieval-grounded metrics, reference-dependent semantic similarity measures, and non-quality operational metrics—to produce a comprehensive and fair comparison between the classical RAG pipeline and the agent-based system.

RAGAS: Reference-Free Retrieval-Grounded Evaluation

The RAGAS framework (Es et al., 2023) provides three metrics that assess the pipeline end-to-end without requiring pre-existing ground truth answer annotations. Context Relevance measures whether retrieved passages are genuinely necessary for answering the query. An LLM is prompted to extract only those sentences from the retrieved context that are essential for a response, yielding a ratio CR $= |S_{\text{extracted}}|/|S_{\text{total}}|$, where a score approaching 1.0 indicates precise, low-noise retrieval. This metric is the most sensitive to retrieval strategy differences between the two systems, though it also shows the weakest alignment with human judgements (0.70 pairwise accuracy), partly due to extraction failures on long passages (Es et al., 2023).

Answer Faithfulness measures the degree to which a generated answer is grounded in the retrieved context, rather than in parametric model knowledge. The metric decomposes the answer into atomic claims, verifies each binary claim against the context, and computes Faithfulness $= |V|/|S|$, where V denotes verified claims and S the total claim count. A score of 1.0 indicates full contextual grounding, while lower scores signal hallucination relative to the retrieved evidence. Validation on the WikiEval benchmark demonstrates 0.95 pairwise accuracy against human judgements, making Faithfulness the most reliable RAGAS metric (Es et al., 2023).

Answer Relevance captures whether the generated response addresses the original query, penalising verbose, incomplete, or off-topic answers. An LLM generates N synthetic paraphrases of the query from the answer text, and the metric computes AR $= \frac{1}{N}\sum_{i=1}^{N} \cos(\mathbf{e}(q), \mathbf{e}(\hat{q}_i))$, where $\mathbf{e}(\cdot)$ denotes the embedding function. Because

RAGAS operates without reference answers, it supports evaluation on synthetic corpora where no pre-existing annotations exist (Es et al., 2023). However, RAGAS cannot detect source routing errors: an answer that is fully grounded in a retrieved passage from the wrong source type still achieves a Faithfulness score of 1.0. Source routing accuracy must therefore be measured as a separate, agent-specific metric.

BERTScore: Reference-Dependent Semantic Similarity

Lexical overlap metrics such as BLEU and ROUGE-L were designed for machine translation and summarisation tasks in which the reference text and the candidate response share significant surface form. In a retrieval setting where correct answers may be expressed through diverse valid paraphrases, BLEU produces near-random correlation with human judgements—reaching a Pearson correlation of -0.019 on the COCO image captioning benchmark (Zhang, Kishore, Wu, Weinberger and Artzi, 2020)—and therefore provides insufficient sensitivity for this evaluation.

BERTScore (Zhang et al., 2020) addresses this gap by computing soft token-level matching between candidate and reference using contextual embeddings from RoBERTa-Large. Recall, precision, and F1 are defined as:

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{i} \max_{j} \mathbf{x}_i^\top \hat{\mathbf{x}}_j \tag{2.2}$$

$$P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{j} \max_{i} \mathbf{x}_i^\top \hat{\mathbf{x}}_j \tag{2.3}$$

$$F_{\text{BERT}} = 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \tag{2.4}$$

where $\mathbf{x}_i$ and $\hat{\mathbf{x}}_j$ are the contextual token embeddings of the reference and candidate respectively, optionally weighted by inverse document frequency to down-weight function words. BERTScore achieves a recall correlation of 0.917 under IDF weighting on the same captioning benchmark, and its advantage over BLEU is largest when paraphrase diversity is high (Zhang et al., 2020)—exactly the condition that characterises open-domain RAG evaluation.

BERTScore and RAGAS are complementary rather than redundant. RAGAS Faithfulness is reference-free and measures grounding in retrieved context; BERTScore is reference-dependent and measures semantic proximity to a correct ground truth

answer. An answer grounded faithfully in incomplete retrieved content may score highly on Faithfulness yet diverge substantially from the correct answer, a gap that only BERTScore captures. Together, the two instruments cover both retrieval-side and generation-side quality without redundancy.

Four Capability Dimensions from Gao et al.

Beyond aggregate metric scores, Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang (2023) identify four robustness dimensions that characterise the qualitative behaviour of retrieval-augmented systems under realistic operating conditions. Noise robustness concerns whether a system can identify and use the relevant passages within a context that also contains irrelevant or contradictory material. Negative rejection tests whether a system correctly declines to answer when no retrieved passage contains useful information, rather than hallucinating a plausible-sounding response. Information integration measures whether the system can synthesise content from multiple retrieved passages to answer multi-hop queries, a capability that single-pass RAG struggles with by design. Counterfactual robustness assesses whether a system can identify and discount factual errors present in retrieved passages. These four dimensions directly inform the query taxonomy used in this evaluation: simple factual queries probe baseline retrieval quality, cross-document synthesis queries test information integration, and ambiguous or incomplete queries probe negative rejection and noise robustness.

Synthetic Query Generation and LLM-as-Judge

No pre-existing annotated query set covers the specific proxy corpus—GitHub Issues, Markdown documentation, and PDF whitepapers—used in this thesis. Synthetic QA generation addresses this constraint within a zero-budget academic context: an LLM is prompted to produce candidate question–answer pairs from sampled document passages, and the resulting set is manually reviewed and filtered against quality criteria before use. RAGAS explicitly validates this methodology, demonstrating that reference-free metrics achieve 0.70–0.95 pairwise accuracy with human judgements and that the framework "enables faster evaluation cycles of RAG architectures without relying on ground truth annotations" (Es et al., 2023). The manual review step is essential for academic credibility, as it filters incoherent, unanswerable, or trivially simple items that

automated generation produces. Ground truth answers produced during this process additionally serve as the reference for BERTScore computation.

Operational and Agent-Specific Metrics

Quality metrics alone are insufficient for an architectural comparison that must inform practical deployment decisions. Latency is measured as wall-clock time from query submission to complete response, averaged over multiple runs, and includes all LLM API calls and, for the agent, all intermediate tool execution steps. Token cost is derived from input and output token counts in API response objects, converted to an estimated cost per query and per 1,000 queries; the agent incurs structurally higher token consumption due to multi-step reasoning traces. Two agent-specific metrics are tracked in addition: tool call count per query (average and stratified by query type) and source routing accuracy, defined as the proportion of queries for which the agent queries the correct source type relative to a manually labelled ground truth. Implementation complexity is assessed through a structured rubric covering flexibility, debuggability, and extensibility, proxied by objective measures including lines of code per component, number of configuration parameters, and the time required to onboard an additional data source. Ferrazzi et al. (2026) explicitly exclude implementation complexity from their evaluation scope; its inclusion here as a formal dimension constitutes a direct contribution of this thesis.

Table 2.1: Evaluation metrics employed in this thesis, organised by dimension.

Dimension	**Metric**	**Reference-free?**
Context relevance	RAGAS Context Relevance	Yes
Answer faithfulness	RAGAS Faithfulness	Yes
Answer relevance	RAGAS Answer Relevance	Yes
Semantic similarity	BERTScore F_1 (RoBERTa-Large)	No
Latency	Mean wall-clock time per query	—
Token cost	API token counts $\rightarrow$ estimated USD	—
Agent efficiency	Tool calls per query	—
Agent routing	Source selection accuracy	No
Complexity	Rubric score + objective proxies	—

Taken together, these considerations suggest that no single metric is sufficient to characterise the difference between a classical RAG pipeline and an agent-based system, and that a multi-dimensional framework spanning retrieval quality, generation fidelity, operational cost, and structural complexity is necessary to produce findings that are both academically rigorous and practically actionable.

3 System Design

3.1 Shared Data Layer

Both the classical RAG pipeline and the agent-based system operate over an identical document corpus. This design decision is fundamental to the comparability of the evaluation: if either system were to ingest different content, differences in retrieval quality could not be attributed to architectural choices alone. The shared data layer therefore serves as the controlled variable in the experimental design, ensuring that all observed differences in latency, quality, and cost arise from the retrieval and generation strategies rather than from corpus coverage.

3.1.1 Data Source Selection and Justification

Enterprise knowledge environments are characterised by structural heterogeneity: relevant information is distributed across issue trackers, collaborative wikis, and document repositories, each imposing different constraints on document length, information density, and query semantics (Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang, 2023). A retrieval system evaluated on documents drawn from a single structural type cannot demonstrate how it performs under this heterogeneity. Three structurally distinct source types are therefore used in this thesis, each selected to serve as a convincing open-source proxy for a category of enterprise documentation.

GitHub Issues as a Jira proxy. Issue tracker entries are short, metadata-rich documents with discrete structured fields—status, labels, assignees—alongside free-text description and comment threads. Closed issues from `kubernetes/kubernetes` and `microsoft/vscode` are used, providing 200 records after filtering for substantive body text (minimum 200 characters) and at least one comment. Closed issues are preferred because they contain resolution context alongside the original problem

description, making them richer sources for question-answer pair generation. The Kubernetes repository provides canonical cloud-native infrastructure issues; the VS Code repository contributes developer-tooling issues, broadening topical coverage within the source type.

Markdown and RST documentation as a Confluence proxy. Collaborative wiki pages are discursive, medium-length documents organised by human-authored headings. Three repositories are cloned using sparse checkout to minimise bandwidth: `kubernetes/website` (English documentation subtree), `ansible/ansible-documentation` (RST format, infrastructure automation), and `cncf/tag-observability` (full clone; includes the 68 KB observability whitepaper and working-group documents). After markup stripping and minimum-length filtering, 1 283 documents are retained.

PDF whitepapers as a SharePoint proxy. Formal document repositories typically contain long, hierarchically structured texts—policy documents, technical reports, architectural specifications. Thirteen CNCF whitepapers and strategy documents are downloaded from the `cncf/tag-security`, `cncf/tag-storage`, `cncf/tag-app-delivery`, `cncf/foundation`, and `cncf/wg-serverless` repositories, yielding 140 extractable sections after discarding graphics-only pages.

Thematic coherence. All three source types are drawn from the cloud-native and Kubernetes ecosystem. This choice is deliberate: shared topical coverage across source types makes cross-document synthesis queries genuinely answerable — a question about Kubernetes security can in principle be addressed by an issue, a documentation page, and a CNCF whitepaper simultaneously. Without thematic overlap, multi-source synthesis queries would be unanswerable by design, removing one of the most important dimensions of the evaluation. Table 3.1 summarises the resulting corpus.

Table 3.1: Corpus statistics after acquisition and filtering.

Source type	Enterprise proxy	Records	Avg. words
GitHub Issues	Jira tickets	200	2 434
Markdown / RST documentation	Confluence wiki	1 283	646
PDF whitepaper sections	SharePoint documents	140	952
Total		**1 623**	

3.1.2 Unified Document Schema

Every record produced by the acquisition pipeline is normalised to a shared JSON schema before it is consumed by either system. Normalisation serves two purposes: it decouples source-specific acquisition logic from the retrieval systems, and it ensures that both the RAG pipeline and the agent-based system receive identically structured inputs, making downstream comparisons valid.

Each record contains five top-level fields. The `source_type` field carries one of three fixed values—`github_issue`, `markdown_doc`, or `pdf_whitepaper`—and is used by the agent's routing logic to identify which retrieval tool to invoke. The `id` field is a globally unique string constructed from the source repository and a document-specific identifier (issue number, file path relative to the repository root, or document name plus section index). The `title` field contains the issue title, the first heading extracted from the document, or the leading text of the first non-junk page of the PDF section. The `content` field contains plain text, stripped of all markup, capped at 8 000 characters to keep token counts manageable during embedding and generation. The `metadata` object carries source-specific fields used for provenance tracing and, in the case of GitHub Issues, for label-based analysis.

Table 3.2: Unified document schema shared by both retrieval systems.

Field	Type	Description
`source_type`	string	`github_issue` | `markdown_doc` | `pdf_whitepaper`
`id`	string	Globally unique document identifier
`title`	string	Document or section heading
`content`	string	Plain-text body, max 8 000 characters
`metadata.repo`	string	Source repository or document name
`metadata.url`	string	Canonical URL of the source
`metadata.labels`	array	Issue labels; empty for docs and PDFs
`metadata.created_at`	string	ISO 8601 timestamp or publication date

The three source types produce separate JSONL files — `github_issues.jsonl`, `markdown_docs.jsonl`, and `pdf_whitepapers.jsonl` — which are read independently by the ingestion components of both systems. The separation by source type is intentional: it allows each system to construct source-specific indexes or tool

backends without requiring a schema change, while the shared field names ensure that logging, evaluation, and post-hoc analysis code operates identically across all three.

3.1.3 Ingestion Data Flow

Acquisition and normalisation proceed in three parallel pipelines, one per source type, before converging at the JSONL files that feed both retrieval systems. Figure 3.1 illustrates the overall flow.

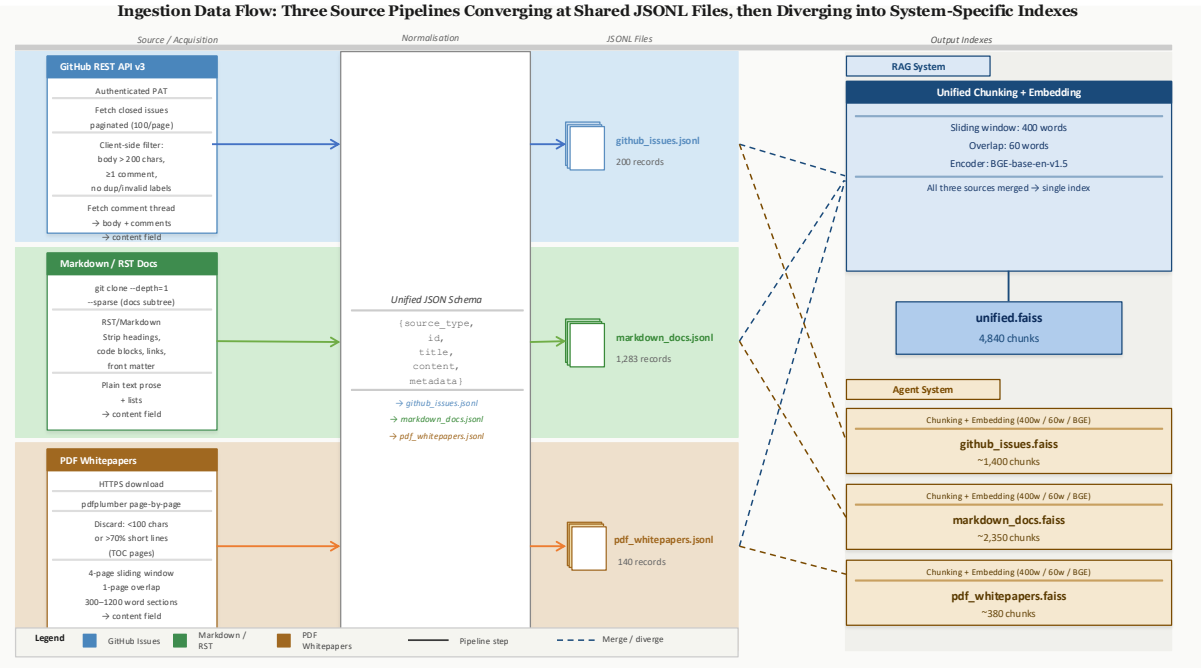


Figure 3.1: Ingestion data flow: three source pipelines converging at shared JSONL files, then diverging into system-specific indexes. Chunking parameters (400-word sliding window, 60-word overlap) and the BGE-base-en-v1.5 embedding model are shared across both systems; the sole architectural difference is a single unified index for RAG versus three per-source indexes for the agent.

GitHub Issues. The GitHub REST API v3 is queried with an authenticated Personal Access Token, raising the rate limit from 60 to 5 000 requests per hour. Closed issues are fetched in pages of 100, filtered client-side against the criteria defined in Section 4, and their comment threads are fetched in a subsequent request per issue. The body and comments are concatenated with a separator and written as a single `content` field.

Markdown and RST documentation. Repositories are cloned with `git clone --depth=1 --filter=blob:none --sparse`, populating only the documentation subtree. This minimises bandwidth and avoids downloading build artefacts, themes, and binary assets. RST files from the Ansible documentation are processed identically to Markdown files; both formats are reduced to plain text by a regular-expression pipeline that strips headings, fenced code blocks, hyperlinks, shortcodes, and front matter, preserving only the readable prose and list content.

PDF whitepapers. PDFs are downloaded via authenticated HTTPS and processed with `pdfplumber`. Text is extracted page by page; pages with fewer than 100 characters of extracted text or exhibiting a table-of-contents structure (more than 70 % of lines shorter than 50 characters) are discarded. The remaining pages are grouped into overlapping four-page windows with a one-page overlap, producing sections of 300–1 200 words that are large enough to contain coherent argumentation yet small enough to fit within a typical retrieval context window.

At the convergence point, both retrieval systems read from the same three JSONL files. The RAG pipeline passes all records through a single unified chunking and embedding stage, producing one vector index. The agent-based system ingests each JSONL file into a separate source-specific index, one per tool. This architectural divergence is one of the key design differences examined in the evaluation.

3.2 Classical RAG Pipeline Architecture

The classical RAG pipeline transforms raw corpus documents into a queryable knowledge store and, at query time, retrieves and synthesises the most relevant passages into a grounded answer. The implementation comprises eight discrete steps organised into two phases: an offline ingestion phase that runs once against the full corpus, and an online query phase that executes on every user request.

3.2.1 Pipeline Stages

Ingestion proceeds sequentially through four steps. Documents are loaded from the three source JSONL files, split into overlapping fixed-size chunks, encoded into dense vector representations by the embedding model, and added to a persistent FAISS

index. At the end of ingestion, 1,623 corpus records have been expanded into 4,840 chunks, each carrying its parent document's source type, title, and metadata. The FAISS index and the chunk metadata store are written to disk and reloaded on startup, so the computationally intensive embedding step is never repeated unless the corpus changes.

The online query phase executes four further steps for each incoming question. The query is encoded by the same embedding model and the resulting vector is compared against all indexed chunk vectors to retrieve the twenty closest candidates. A cross-encoder reranker then scores each candidate against the query and returns the five highest-scoring chunks. These chunks are assembled into a numbered context block and passed to the language model alongside a system prompt that instructs the model to answer using only the supplied passages. Latency and token counts are captured for every query and appended to a structured JSONL log, producing the measurement records used in Chapter 6.

3.2.2 Chunking Strategy

Each corpus document is split into overlapping windows of 400 words with a 60-word overlap between consecutive windows. The step size between window starts is therefore 340 words, meaning that the first 60 words of each chunk also appear at the end of the preceding chunk. This overlap preserves semantic continuity across boundaries: a sentence or argument that spans a chunk boundary is fully represented in at least one chunk, ensuring that a relevant passage is not fragmented out of the index (Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang, 2023).

Chunking operates on whitespace-delimited word lists rather than on subword tokens produced by a transformer tokeniser. This makes the chunking step independent of any specific model vocabulary, so that the embedding or reranker model can be replaced without recalibrating chunk sizes. In practice, a 400-word chunk corresponds to approximately 480–550 subword tokens for English technical prose, which falls within the 512-token context limit of the embedding model and well below the 8,192-token limit of the cross-encoder.

The 400-word target balances retrieval precision against contextual coherence, a trade-off documented explicitly in the Advanced RAG literature (Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang, 2023). Shorter chunks increase the likelihood that a retrieved

passage is topically homogeneous, but risk fragmenting multi-step arguments that require surrounding context to be interpretable; longer chunks preserve coherence but dilute the relevance signal. The 60-word overlap and 400-word window together place this implementation within the Advanced RAG category of the Naive / Advanced / Modular taxonomy, distinguishing it from Naive RAG pipelines that apply fixed splits without overlap.

3.2.3 Embedding Model and Vector Store

Chunks and queries are encoded using `BAAI/bge-base-en-v1.5`, a 768-dimensional bi-encoder model executed locally via the `sentence-transformers` library. The model was selected from the MTEB Retrieval leaderboard as a top-performing open-weight model at its parameter scale, requiring no external API call and incurring no per-token cost. BGE models are trained with an asymmetric query–document encoding scheme: at query time, the string *"Represent this sentence for searching relevant passages:"* is prepended to the raw query before encoding, while document chunks are encoded without any prefix. This asymmetry is required for the model to produce embeddings that align query intent with document content; omitting the prefix at query time measurably degrades retrieval quality (Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang, 2023).

All embeddings are L2-normalised to unit length before any similarity computation, both at ingestion time and at query time. Normalisation allows the FAISS `IndexFlatIP` index to compute cosine similarity via the inner product: for unit-length vectors $\mathbf{u}$ and $\mathbf{v}$, the relation $\mathbf{u} \cdot \mathbf{v} = \cos\theta$ holds, making inner-product maximisation equivalent to cosine similarity maximisation. Cosine similarity is preferred over raw inner product because it is magnitude-invariant — a longer chunk does not receive a spuriously high score simply by virtue of producing a larger embedding vector.

FAISS `IndexFlatIP` performs exact nearest-neighbour search over the full index without quantisation or approximation. At 4,840 vectors of dimension 768 stored as 32-bit floats, the index occupies approximately 14 MB of memory and returns query results in under one second on the development hardware. Exact search is therefore practical at this corpus scale and avoids the recall penalty introduced by approximate methods such as HNSW or IVF clustering, which trade retrieval accuracy for speed at larger scales (Lewis et al., 2020).

3.2.4 Retrieval and Reranking

Retrieval proceeds in two stages to exploit the complementary properties of bi-encoders and cross-encoders. In the first stage, the FAISS index returns the twenty document chunks whose embeddings have the highest cosine similarity to the embedded query. These twenty candidates are then passed to `cross-encoder/ms-marco-MiniLM-L-6-v2`, a cross-encoder model fine-tuned on the MS MARCO passage-ranking benchmark, which scores each query–chunk pair jointly by processing the concatenated pair as a single transformer input.

The key distinction between the two stages lies in how query and document representations are produced. Bi-encoders encode queries and documents independently and compare them post hoc; this enables sub-second retrieval over arbitrarily large indexes but prevents the model from attending to fine-grained token-level interactions between the two texts. Cross-encoders process both texts as a single input, allowing full cross-attention across the query–chunk boundary and producing substantially more accurate relevance scores at the cost of one forward pass per candidate (Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang, 2023). Applying the cross-encoder only to the twenty FAISS candidates bounds its computational cost to a fixed and small number of inference calls regardless of index size, keeping online latency predictable.

The five highest-scoring chunks after reranking are passed to the generator. Each chunk carries both its FAISS cosine similarity score and its cross-encoder rerank score in the log record, providing a complete provenance trail for post-hoc analysis of retrieval behaviour. Temperature is set to zero for all generation calls to ensure fully deterministic outputs across evaluation runs, a requirement for the reproducibility of the evaluation described in Chapter 4.

3.3 Agent-Based System Architecture

The agent-based system replaces the RAG pipeline's fixed retrieve-then-generate sequence with an iterative reasoning loop in which the language model selects and invokes retrieval tools based on the content and requirements of each query. Rather than encoding the entire corpus into a single unified index, the system exposes three source-specific retrieval tools and delegates the decision of which tool to call, in what

order, and with what query formulation to the language model at runtime. The implementation follows the ReAct framework (Yao et al., 2023) and is built using LangChain's agent scaffolding on top of the same language model endpoint and embedding infrastructure used by the RAG pipeline.

3.3.1 The Agent Loop

ReAct defines an interleaved reasoning and acting pattern in which a language model alternates between generating a *Thought*—an explicit natural-language reasoning step—and taking an *Action*—a structured tool call with explicit inputs (Yao et al., 2023). After each action, the tool's return value is appended to the context as an *Observation*, and the model continues reasoning from the updated context until it emits a final answer prefixed with the token *Final Answer*. This cycle constitutes a closed-loop process: unlike the single retrieval pass of the RAG pipeline, the agent can inspect retrieved evidence, judge its sufficiency, and issue further tool calls or reformulated queries before committing to a response.

The implementation uses LangChain's `create_react_agent` function, which binds the language model, the registered tool set, and a ReAct-format prompt into an executable agent definition, and `AgentExecutor`, which drives the reasoning cycle at runtime. The language model is accessed through LangChain's `ChatOpenAI` wrapper pointed at the same OpenAI-compatible endpoint used by the RAG pipeline—Ollama in development, AI Refinery during evaluation—with identical model identifier and temperature settings. A maximum iteration count of fifteen is enforced by the `AgentExecutor`; this bound accommodates queries that require all three tools in sequence while preventing unbounded loop growth on ambiguous or unanswerable inputs.

3.3.2 Tools and Source Routing

Three retrieval tools are registered with the agent, one per corpus source type. Each tool wraps a dedicated FAISS index constructed from the same corpus using the same 400-word sliding-window chunking, 60-word overlap, and BGE-base-en-v1.5 embedding model as the RAG pipeline. Separate per-source indexes are used rather than the RAG pipeline's unified index to reflect the architectural reality of enterprise knowledge

environments, in which issue trackers, documentation platforms, and document repositories are independent systems with distinct search interfaces (Ferrazzi et al., 2026). An agent operating over a unified index would have no structural basis on which to make source-routing decisions; partitioning the indexes makes routing a genuine runtime choice with observable consequences for retrieval quality and, consequently, answer faithfulness.

Table 3.3: Agent retrieval tools and their source-type scope.

Tool name	**Source type**	**Enterprise proxy**
`search_github_issues`	GitHub Issues	Jira tickets
`search_markdown_docs`	Markdown/RST docs	Confluence wiki
`search_pdf_whitepapers`	CNCF PDF sections	SharePoint documents

Each tool applies the same two-stage retrieval used in the RAG pipeline: twenty nearest-neighbour candidates are retrieved from the tool's FAISS index and then reranked by `cross-encoder/ms-marco-MiniLM-L-6-v2` to a top-five result set. This ensures that retrieval quality per tool call is directly comparable to the RAG pipeline's retrieval quality, so that any differences in final answer quality reflect the agent's routing and synthesis behaviour rather than differences in the underlying retrieval mechanics.

3.3.3 Orchestration Logic

Source routing is implemented entirely through the natural-language descriptions attached to each tool in the ReAct system prompt. No explicit rule-based dispatch logic is implemented; the language model reads each tool's description and selects the appropriate tool based on query semantics at each reasoning step. The descriptions are written to reflect the structural characteristics and dominant query patterns of each source type: the GitHub Issues tool is described as appropriate for bug reports, error messages, and issue resolutions; the documentation tool for conceptual explanations, how-to guides, and configuration references; the whitepaper tool for standards, best practice recommendations, and maturity frameworks. This design is authentic to the ReAct pattern, in which tool selection is part of the Thought generation step (Yao et al., 2023), and avoids brittle keyword-based dispatch that would require maintenance whenever the corpus is extended.

Tool observations are returned as formatted plain text containing the five reranked chunks, each prefixed with its source type, document title, and cosine similarity score. This format supplies the language model with the provenance information required to attribute claims and to judge whether a further tool call is warranted, while keeping each observation compact enough to avoid exhausting the context window across multi-step queries. The agent may call the same tool more than once with a reformulated query if the first observation is insufficient, or it may call multiple tools in sequence for cross-source synthesis queries.

All tool calls, their inputs, and their observations are recorded in a structured `agent_trace` field appended to each log record alongside the query, final answer, total latency, and token counts. The log schema is a strict superset of the RAG pipeline's schema: all shared fields—query, answer, latency, token counts, and retrieved chunks—are produced identically by both systems, while the `agent_trace`, tool-call count, and source-routing fields are present only in agent logs. This shared outer schema ensures that the evaluation infrastructure processes both systems' outputs without modification.

3.3.4 Multi-Agent Coordination Architecture

The multi-agent architecture was designed as a corrective response to structural failure modes identified during single-agent evaluation: context accumulation across the ReAct loop exhausted the iteration ceiling, causing 41 per cent of queries to fail before a final answer was produced. Rather than lengthening the iteration limit further, the architecture eliminates the loop entirely.

The design is a two-node LangGraph `StateGraph`. The first node, `retrieve_all_sources`, queries all three source-specific FAISS indexes sequentially — GitHub Issues, Markdown documentation, and PDF whitepapers — applying the same cross-encoder reranking pipeline used in both the RAG and single-agent systems to each source in turn, and accumulating at most fifteen chunks in total, five per source. The second node, `synthesize`, receives the pooled passages as a single bounded context window and issues exactly one language model call to produce the final answer. No further retrieval is possible once synthesis begins; the context presented to the model is fixed at the point the first node completes.

Three design properties distinguish this architecture from the single-agent system. There is no reasoning loop and therefore no iteration limit and no risk of `GraphRecursionError`. The context window per query is fixed at fifteen chunks regardless of query type or complexity, making the per-query token budget fully predictable. All three sources are queried on every request, so source coverage is an architectural guarantee rather than a model-driven decision (Xi et al., 2023). These properties come at a cost: the multi-agent system cannot direct retrieval effort selectively toward the most relevant source, nor can it reformulate a query after inspecting an initial observation.

The multi-agent reuses the per-source FAISS indexes constructed by `agent/ingest.py` and the `SourceRetriever` class from `agent/tools.py`, adding no new ingestion infrastructure. The primary trade-off relative to the single agent is coverage guarantee versus query-adaptive selectivity: the multi-agent provides complete source coverage at the cost of the ability to concentrate retrieval on a single source for queries whose answer is localised to one part of the corpus.

3.4 Comparability Design Decisions

A valid architectural comparison requires that observed differences in evaluation outcomes be attributable to the retrieval and orchestration strategies under study rather than to differences in data, models, or hardware. Every component not itself under comparison is therefore held constant across both systems. Table 3.4 lists the shared and varied parameters; the design decisions implicit in each row are elaborated below.

3.4.1 Shared Experimental Parameters

The language model, embedding model, chunking parameters, and reranker are identical across both systems. Both access the language model through the same OpenAI-compatible endpoint—Ollama locally, AI Refinery for evaluation—using the same model identifier and a generation temperature of zero. Temperature is fixed at zero to produce fully deterministic outputs; any repeated query to either system yields the same response, which is a prerequisite for reliable latency averaging and reproducible RAGAS scoring. Both systems embed queries and corpus chunks using

`BAAI/bge-base-en-v1.5` with L2-normalised vectors and the same retrieval-prefix convention, and both apply `cross-encoder/ms-marco-MiniLM-L-6-v2` to a candidate pool of twenty, returning five chunks as the final retrieved context.

Table 3.4: Controlled and varied parameters across both systems.

Parameter	**RAG Pipeline**	**Agent System**
Language model	shared endpoint; swapped via `.env`	shared endpoint; swapped via `.env`
Embedding model	BGE-base-en-v1.5 (768d)	BGE-base-en-v1.5 (768d)
Chunking	400 words, 60-word overlap	400 words, 60-word overlap
Reranker	MiniLM, top-5	MiniLM, top-5 per tool call
Document corpus	shared JSONL source files	shared JSONL source files
Generation temperature	0 (deterministic)	0 (deterministic)
Hardware and runtime	local CPU, single venv	local CPU, single venv
Index structure	Unified FAISS (4 840 chunks)	Three per-source FAISS indexes
Query resolution	Fixed 4-step pipeline	Iterative loop, max 15 steps
Source routing	Implicit (unified index)	Explicit (LLM tool selection)

The document corpus is shared at the source level: both systems read from the same three JSONL files in `data/corpus/`. Their index structures differ because the RAG pipeline consolidates all records into a single FAISS index while the agent builds three separate per-source indexes from the same files. The total chunk count differs marginally—4,840 in the unified index versus approximately 4,130 across the three agent indexes—because the Markdown corpus is lightly downsampled during agent ingestion to keep index sizes balanced. Both systems run in the same Python virtual environment on the same hardware, eliminating interpreter version, library dependency, and process-level differences as confounds in the latency measurements.

3.4.2 Defining Equivalent Effort

Implementation complexity is evaluated as a first-class metric in this thesis, corresponding to Research Question 3. To make this comparison defensible, complexity

is operationalised through three observable proxies rather than through subjective judgements about code quality or design elegance.

Lines of non-blank, non-comment code are counted per component for four parallel components in each system: ingestion, retrieval or tools, generation or agent executor, and the pipeline orchestrator. Framework calls—including LangChain agent and tool definitions—count toward the agent's total, because they represent integration and configuration work a developer must write and maintain. The second proxy is the number of explicit configuration parameters in each system's `config.py` file, reflecting the number of decisions a practitioner must make to deploy or tune the system. The third proxy is the number of steps required to onboard a new data source: for the RAG pipeline this involves appending records to the unified ingestion script and re-running the embedding step; for the agent it additionally requires building a new per-source FAISS index, writing a new `@tool` function, and updating the tool descriptions in the system prompt. These three proxies together provide an objective basis for the rubric-based complexity scores presented in Chapter 6.

4 Data and Evaluation Setup

4.1 Dataset Construction

The corpus used in this evaluation was assembled from three structurally distinct open-source document collections, each selected as a proxy for a category of enterprise knowledge source. Collection, preprocessing, and normalisation were automated through three Python scripts — `01_fetch_github_issues.py`, `02_fetch_markdown_docs.py`, and `03_fetch_pdfs.py` — executed against publicly accessible repositories and APIs. All records were written to the unified JSONL schema defined in Section 3 before being consumed by either retrieval system.

4.1.1 Data Collection per Source Type

GitHub Issues. Issues were retrieved from `kubernetes/kubernetes` and `microsoft/vscode` via the GitHub REST API v3 using a Personal Access Token, which raises the authenticated rate limit from 60 to 5 000 requests per hour. A target of 100 qualifying issues per repository was set, yielding a maximum corpus of 200 records. Four filter criteria were applied during collection: only closed issues were fetched, because closed records contain resolution context alongside the original problem description; the issue body was required to exceed 200 characters, excluding stub entries; at least one comment was required, ensuring discussion beyond the opening post; and issues carrying the labels *duplicate*, *invalid*, or *wontfix* were discarded. Pull requests, which share the GitHub issues endpoint, were identified and excluded via the presence of a `pull_request` field in the API response. For each qualifying issue, a separate request retrieved up to 50 comments; the issue body and all comment texts were concatenated with a separator into a single `content` field.

Markdown and RST documentation. Three repositories were cloned using `git clone --depth=1 --filter=blob:none --sparse` to minimise bandwidth by downloading only the relevant documentation subtree. From `kubernetes/website`, only the `content/en/docs` directory was checked out, yielding Markdown files from the official Kubernetes documentation. From `ansible/antsible-documentation`, the `docs/` directory was checked out, containing both Markdown and RST files. The `cncf/tag-observability` repository was cloned in full, as it is a small repository whose content — including the 68 KB observability whitepaper and working-group documents — is distributed across the root and subdirectories. Directories that contain navigation scaffolding, theme assets, and non-prose content were excluded by path matching against a fixed skip-list (`assets`, `static`, `themes`, `_layouts`, and equivalents). Files smaller than 1 KB were discarded before markup stripping.

PDF whitepapers. Fourteen CNCF documents were downloaded via HTTPS from five repositories: `cncf/tag-security` (five documents, `main` branch), `cncf/tag-storage` (two documents, `master` branch), `cncf/tag-app-delivery` (three documents), `cncf/foundation` (two documents), and `cncf/wg-serverless` (one document, `master` branch). Text was extracted page by page using `pdfplumber`. Pages were classified as junk and discarded if they contained fewer than 100 characters of extracted text or if more than 70 % of their non-empty lines were shorter than 50 characters — a heuristic that reliably identifies cover pages and table-of-contents pages without requiring heading detection. The CNCF AI-assisted development survey yielded zero extractable text pages due to a graphics-only layout and was excluded from the corpus. The remaining 13 documents were segmented into overlapping four-page windows with a one-page overlap, producing sections of coherent prose suitable for retrieval.

4.1.2 Preprocessing and Normalisation

Markdown and RST files were reduced to plain text by a regular-expression pipeline that removed YAML front matter, fenced code blocks, inline code spans, HTML tags, Hugo shortcodes, and Markdown image references. Markdown hyperlinks were reduced to their display label; RST roles were similarly reduced to their content. RST structural underline characters and directive declarations were removed. Consecutive blank lines were collapsed to a single blank line. This pipeline was applied uniformly to both Markdown and RST files; no format-specific branch was required beyond the

removal of RST-specific syntax. All records from the documentation and PDF sources were capped at 8 000 characters after stripping, preventing any single document from dominating retrieval contexts. GitHub issue content was not capped at this stage, as issue bodies and comment threads are segmented into 400-word overlapping chunks during index construction.

Every record produced by each acquisition script was written to the same five-field JSONL schema — `source_type`, `id`, `title`, `content`, `metadata` — described in Section 3. No cross-source deduplication was performed, as the three source types draw from non-overlapping repositories with non-overlapping document namespaces. Within each source, uniqueness is guaranteed by the issue number (GitHub), the file path relative to the repository root (Markdown), and the document name plus section index (PDF).

4.1.3 Corpus Statistics

The completed corpus contains 1 623 records across the three source types, with a combined total of approximately 1 449 000 words. Table 4.1 reports per-source record counts, word-length statistics, and estimated token counts; the token estimates assume an average of 1.3 subword tokens per English word, consistent with the BGE tokeniser's subword vocabulary for technical prose.

Table 4.1: Corpus statistics after acquisition, filtering, and normalisation.

Source type	Records	Avg words	Total words	Approx. tokens
GitHub Issues	200	2 434	486 826	633 000
Markdown / RST docs	1 283	646	829 088	1 078 000
PDF whitepaper sections	140	952	133 281	173 000
Total	**1 623**		**1 449 195**	**1 884 000**

GitHub issues show the widest length variation: the distribution spans from brief issue stubs that passed the 200-character body filter to multi-comment threads exceeding 2 000 words, with the majority falling between 200 and 500 words after the body and comments are concatenated. Markdown documents are tightly bounded by the 8 000-character content cap, producing a maximum of approximately 1 400 words per record and a distribution concentrated between 500 and 1 000 words. PDF sections, also capped

at 8 000 characters, are the most length-uniform source: 88 of 140 sections fall in the 1 000–2 000-word range, reflecting the dense prose typical of formal technical whitepapers. The length diversity across source types mirrors the structural heterogeneity of the enterprise environment this corpus is designed to simulate.

4.2 Evaluation Query Set

4.2.1 Synthetic QA Generation

Candidate question–answer pairs are generated by prompting `gpt-oss-120b`, accessed through the Accenture AI Refinery endpoint, against corpus excerpts sampled from the indexes built in Section 4.1. This LLM-assisted approach is established practice for constructing domain-specific evaluation benchmarks when no annotated dataset exists (Es et al., 2023). Using the same model for generation and for answer evaluation, rather than a separate weaker generator, keeps phrasing conventions consistent between ground-truth answers and the system outputs under evaluation, which reduces spurious penalties in semantic similarity scoring.

One structured prompt template was designed for each query type; all templates instruct the model to return a JSON object containing the question, the ground-truth answer, and the list of source chunk identifiers forming the evidence base. The prompt for factual and synthesis questions specifies three requirements: the question must be answerable solely from the provided passage or passages, the wording must not quote or paraphrase the answer, and the question must be specific enough to admit exactly one correct answer. Responses are parsed with `json.loads`; responses that fail to parse or violate the expected schema are discarded before human review. The generation script, `scripts/evaluation/generate_queries.py`, handles chunk sampling, prompt construction, API calls, and result storage. All generated candidates are written to `data/eval/candidates.jsonl`; the final reviewed set is stored in `data/eval/queries.jsonl` under the schema defined in Table 4.3.

4.2.2 Query Taxonomy

The evaluation set contains 90 queries organised into four types, each designed to stress one of Gao et al.'s four RAG capability dimensions (Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang, 2023). Table 4.2 summarises the taxonomy. Structuring the query set around these dimensions makes the comparison between systems directly interpretable in terms of the literature: performance differences on each subset can be attributed to a specific aspect of retrieval behaviour rather than to undifferentiated query difficulty.

Table 4.2: Evaluation query taxonomy: four types mapping to Gao et al.'s RAG capability dimensions.

Type	Capability dimension	Count	Description
Factual	Noise robustness	30	Single-chunk answer; one source type
Synthesis	Information integration	30	Answer spans 2+ passages; $\geq$10 cross-source
Ambiguous	Noise robustness	15	Underspecified query; no unique correct answer
Unanswerable	Negative rejection	15	Plausible but corpus-absent topic
Total		**90**	

Factual queries each have a single correct answer contained within one corpus chunk. The generation script samples chunks stratified by source type—17 from Markdown documentation, 17 from GitHub issues, and 16 from PDF whitepapers—and prompts the model to produce one factual question per chunk. After manual review, the accepted set is balanced to approximately 10 questions per source type.

Synthesis queries require combining information from at least two passages to answer correctly. Chunk pairs were sampled in three configurations: pairs from within the same document, pairs from different documents of the same source type, and pairs spanning two distinct source types (cross-source). Approximately 12 of the 40 sampled pairs are cross-source; the review process requires that at least 10 cross-source questions survive to the final set. Cross-source synthesis queries are structurally important for the agent evaluation: they require the agent to invoke multiple tools rather than resolving the query from a single index, making routing accuracy directly observable.

Ambiguous queries are derived from accepted factual questions by removing one key identifying term—the technology name, the version specifier, or the system being described—and prompting the model to rewrite the result as a naturally underspecified query. The rewritten question remains plausible and domain-relevant but lacks the context required to return a unique correct answer. Approximately five ambiguous queries target each source type.

Unanswerable queries are composed by hand rather than generated by a language model, because a model prompted against corpus excerpts cannot reliably invent plausible questions about topics the corpus does not contain. Each unanswerable question concerns a topic adjacent to the corpus domain but absent from every source—for example, Kubernetes features introduced after the issue snapshot date, security standards not referenced in any whitepaper, or Ansible modules absent from the documentation clone. Each candidate is verified by running it through both retrieval systems and confirming that no top-5 retrieved chunk provides a sufficient answer.

4.2.3 Manual Review and Filtering

Every LLM-generated candidate undergoes manual review before acceptance into the final query set. A candidate is accepted if six criteria are satisfied: the ground-truth answer is directly verifiable against the cited source text without inference beyond what is written; the question wording does not quote or paraphrase the answer; the question is non-trivial in the sense that a keyword match on its surface terms is insufficient to locate the answer; for synthesis candidates, the answer cannot be produced from any single chunk in isolation and both cited passages are genuinely required; for ambiguous candidates, the rewritten question reads naturally and would plausibly be asked by a domain non-expert; and the English is clear, well-formed, and domain-appropriate. A candidate is rejected if its answer admits multiple valid interpretations, if the question is a near-verbatim copy of a sentence from the source text, or if a correct answer requires domain knowledge not present in the corpus.

Rejected candidates are deleted from `data/eval/candidates.jsonl`; accepted candidates may be edited before transfer to `data/eval/queries.jsonl`. A validation script, `scripts/evaluation/check_queries.py`, enforces schema compliance and reports per-type counts after review, confirming that the final set meets the distribution targets in Table 4.2.

4.2.4 Final Query Set Statistics

Table 4.3: Query set schema for `data/eval/queries.jsonl`.

Field	Type	Description
`id`	string	Unique query identifier (`q001`–`q090`)
`question`	string	The evaluation question
`ground_truth`	string	Correct answer verifiable against source text
`query_type`	enum	`factual`, `synthesis`, `ambiguous`, `unanswerable`
`source_type`	enum	`github_issue`, `markdown_doc`, `pdf_whitepaper`, `cross_source`, `none`
`source_ids`	string list	Chunk identifiers forming the ground-truth evidence base
`notes`	string	Manual annotation (e.g., ambiguity rationale, verification notes)

The final set comprises 90 queries distributed across the four types and three primary source categories shown in Table 4.2. The `source_type` field serves a dual purpose: it provides the primary expected source category for human-readable taxonomy, and it functions as the routing ground-truth label for agent evaluation. A query is considered correctly routed if at least one tool call in the agent trace targets the expected source type; for `cross_source` queries, both expected source types must appear in the agent's `sources_used` log field. The `source_ids` field identifies the specific chunks that constitute the evidence base for each question, enabling post-hoc verification of context relevance at the chunk level during RAGAS evaluation. Together, these two fields provide the ground-truth annotations required to evaluate both systems on retrieval quality and, for the agent specifically, on source routing accuracy.

4.3 Evaluation Metrics Definition

4.3.1 RAGAS Scores

The three RAGAS metrics defined by Es et al. (2023) — context relevance, faithfulness, and answer relevance — form the primary quality signal for both systems, each tar-

geting a distinct failure mode of RAG. Context relevance measures how much of the retrieved context is relevant to the query: a low score indicates that the retrieval stage surfaces noisy or off-topic passages that dilute the context window. Faithfulness measures the fraction of factual claims in the generated answer that are directly supported by the retrieved context: a low score signals that the model asserts information not grounded in the retrieved material. Answer relevance measures how well the generated answer addresses the original question, scored by embedding multiple questions reverse-generated from the answer and computing their mean cosine similarity to the query: a low score indicates that the answer, while potentially faithful to the context, fails to respond to what was actually asked.

All three metrics are computed by the RAGAS library using an LLM judge, which is invoked internally to classify retrieved sentences as relevant or irrelevant, to extract answer claims and verify them against the retrieved context, and to generate the reverse questions used in answer relevance scoring. Each metric requires four inputs per query: the question, the generated answer, the list of retrieved context passages, and — for faithfulness and answer relevance — the ground-truth answer string. These inputs are assembled from two sources: the question and ground-truth fields from `data/eval/queries.jsonl`, and the answer and retrieved chunk texts from the pipeline log.

The RAGAS judge is configured to call `gpt-oss-120b` via the Accenture AI Refinery endpoint — the same model used to generate answers in both evaluation systems. Using the same model as both generator and judge prevents the judge's linguistic conventions from diverging from those of the generator, which would introduce a systematic bias into faithfulness and answer relevance scoring. Because the RAG and agent logs share an identical outer schema, the RAGAS input construction script applies without modification to either log file, ensuring that metric computation is structurally equivalent for both systems.

4.3.2 BERTScore and ROUGE-L

BERTScore computes precision, recall, and F1 between a generated answer and a ground-truth reference by matching contextual token embeddings rather than surface word forms, which allows it to credit semantically equivalent phrasings that exact-match metrics would penalise (Zhang et al., 2020). The F1 score is the reported figure.

Embeddings are produced by `microsoft/deberta-xlarge-mnli` with baseline rescaling, the recommended configuration for English-language evaluation, which calibrates raw cosine similarity scores against a language-model baseline to improve cross-domain comparability.

ROUGE-L measures the longest common subsequence (LCS) between the generated and reference answer, normalised by reference length to produce a recall-oriented score. No model inference is required: the metric is computed using the standard `rouge-score` Python library and is fully deterministic. ROUGE-L is sensitive to word order and exact token overlap, making it a conservative lower bound on quality relative to BERTScore.

Both metrics are applied to the `answer` field from the pipeline log against the `ground_truth` field from `data/eval/queries.jsonl` across all 90 queries for both systems. Neither serves as a primary decision criterion for the comparison in Chapter 6: RAGAS captures the retrieval-specific quality properties — faithfulness and context relevance — that string-based metrics cannot directly observe. BERTScore and ROUGE-L provide secondary corroboration and offer a reference point for readers familiar with standard natural language generation benchmarks.

4.3.3 Latency and Token Cost

Latency and token consumption are read directly from the structured logs that both pipeline implementations produce, without requiring additional instrumentation. Response latency is recorded as `latency.total_s` — the wall-clock time from query submission to the point at which the final answer string is complete. For the RAG pipeline this interval spans FAISS nearest-neighbour search, cross-encoder reranking, and LLM generation. For the agent pipeline it spans the entire ReAct loop, including every tool invocation and the final synthesis call. Both are measured end-to-end on the same machine, calling the same AI Refinery API endpoint, so hardware and network conditions are identical across systems.

Reported latency statistics include the mean, median, and standard deviation across all 90 queries, and a per-query-type breakdown — factual, synthesis, ambiguous, and unanswerable — to reveal where each system's timing profile diverges. Token consumption is read from `tokens.input` and `tokens.output` in the log, captured via the OpenAI API response object for RAG and via `get_openai_callback()`

for the agent. Because the AI Refinery endpoint does not publish per-token pricing externally, raw token counts form the primary reported figure. A cost estimate in USD is also provided using the publicly available GPT-4o pricing schedule ($2.50 per million input tokens, $10.00 per million output tokens) as a proxy, and is explicitly labelled as an approximation.

Token cost is a practically important dimension for enterprise deployments. The agent's multi-step ReAct loop accumulates context across multiple LLM calls per query, typically producing higher input-token counts than the RAG pipeline's single generation call. At enterprise query volumes, the accumulated cost of additional tool-call context can outweigh retrieval quality gains for cost-sensitive teams, making this metric central to the practical trade-off analysis in Chapter 7.

4.3.4 Agent-Specific Metrics

Two metrics apply exclusively to the agent system and are computed post-hoc from the agent log: source routing accuracy and mean tool call count per query. Routing accuracy measures the fraction of queries for which the agent's tool-call pattern matches the ground-truth source type specified in `data/eval/queries.jsonl`. For queries with a single expected source type — factual, within-source synthesis, and ambiguous — a query is counted as correctly routed if at least one entry in the `sources_used` log field matches the expected `source_type` label. For cross-source synthesis queries, correct routing requires that both expected source types appear in `sources_used`. Unanswerable queries are excluded from routing accuracy computation, as no source type is defined for them.

Mean tool call count is the average value of the `tool_calls` field in the agent log, reported both as an overall mean and broken down by query type. The per-type breakdown reveals characteristic routing patterns: factual queries are expected to average approximately one tool call, cross-source synthesis queries approximately two, and unanswerable queries expose whether the agent searches exhaustively before declining. A high mean tool call count on unanswerable queries indicates that the agent is investing retrieval effort on queries that no source can resolve, rather than recognising the absence of relevant material and declining early.

Neither metric applies to the RAG pipeline, which has no routing step and no iterative loop: RAG queries the unified index with every request, and its log records a tool call

count of zero by construction. These metrics are therefore reported in a dedicated agent-only subsection of Chapter 6, separate from the cross-system comparison tables.

4.3.5 Complexity and Maintainability Rubric

Implementation complexity and maintainability cannot be captured by automated metrics and are assessed using a structured rubric applied to both systems by the author on the basis of implementation experience. The rubric evaluates five criteria chosen to reflect the concerns of an enterprise team making an architectural decision between the two approaches: implementation effort, configurability, debuggability, extensibility, and operational overhead. Table 4.4 defines each criterion and the objective proxy used to anchor the score.

Table 4.4: Complexity and maintainability rubric: five criteria scored 1–5 (5 = simpler / lower complexity).

Criterion	**What it measures**	**Objective proxy**
Implementation effort	Code volume and time required to build from scratch	Lines of code per functional component
Configurability	Parameters required to tune for a new deployment	Parameter count in `config.py`
Debuggability	Ease of tracing a failure to its root cause	Failure modes identified during testing; call-stack depth at error
Extensibility	Effort to onboard a new data source	Steps required to add one new source type
Operational overhead	Runtime dependencies and services required	Dependency count; external services at runtime

Each criterion is scored from one to five, where five denotes lower complexity or greater ease of use. Scores are assigned by the author and justified with the corresponding objective proxy where one is available; where a proxy does not fully determine the score, the rationale is stated explicitly. Lines of code are counted with blank lines and comment-only lines excluded, reported per functional component — ingest, retrieval or tools, generation or agent, and pipeline. The rubric produces a single comparison table in Chapter 6 that reports per-criterion scores and an unweighted mean for each system.

The mean is treated as indicative rather than definitive, given the small criterion set and single evaluator — a limitation acknowledged in Chapter 7.

5 Implementation

5.1 Development Environment and Tooling

Both systems are implemented and evaluated on an Apple M4 workstation with 16 GB of unified memory, running macOS. All code is written in Python 3.14.6. No discrete GPU is required: `sentence-transformers` version 5.x automatically selects Apple Silicon MPS as the compute backend for embedding and cross-encoder inference on M4, providing hardware acceleration without CUDA. FAISS nearest-neighbour search runs on CPU using `faiss-cpu`, which is sufficient for the index sizes used — 4,840 vectors in the unified RAG index and at most 1,499 vectors per agent sub-index.

Table 5.1 lists all dependencies with confirmed installed versions. The dependency set is partitioned into two requirements files: `requirements_rag.txt` covers the RAG pipeline alone, and `requirements_agent.txt` covers the agent and is a strict superset of the RAG requirements. Installing the agent requirements file is therefore sufficient to prepare both systems for execution. The evaluation libraries — RAGAS, BERTScore, and rouge-score — are not installed during development to avoid dependency conflicts with the inference stack; they are added to the same environment immediately before the evaluation scripts are run.

The repository is organised into four top-level directories — `rag/`, `agent/`, `data/`, and `scripts/` — each encapsulating one functional concern. The listing below shows the layout at two levels of depth, with the large corpus and index directories collapsed.

```
.
├── .env                    API keys and model name (not committed)
├── .env.example            Template with placeholder values
├── requirements_rag.txt    RAG pipeline dependencies
├── requirements_agent.txt  Agent dependencies (superset of RAG)
├── data/
```

```
│   ├── corpus/              Three JSONL files, one per source type
│   ├── eval/                queries.jsonl; candidates.jsonl
│   └── raw/                 Downloaded PDFs; cloned doc repositories
├── rag/
│   ├── config.py            Models, chunk size, top-k, prompts
│   ├── ingest.py            Builds unified FAISS index from corpus
│   ├── retrieve.py          FAISS search + CrossEncoder reranking
│   ├── generate.py          AI Refinery LLM call
│   ├── pipeline.py          Orchestrate retrieve + generate; log
│   └── indexes/             unified.faiss + unified_chunks.jsonl
├── agent/
│   ├── config.py            LLM, top-k, iterations, system prompt
│   ├── ingest.py            Builds three per-source FAISS indexes
│   ├── tools.py             @tool functions; per-source retrieval
│   ├── agent.py             create_react_agent construction
│   ├── pipeline.py          Run ReAct loop; log
│   └── indexes/             Per-source .faiss + _chunks.jsonl
├── multi_agent/
│   ├── config.py            LLM settings, synthesis prompt, log dir
│   ├── graph.py             LangGraph StateGraph (retrieve + synthesize)
│   └── pipeline.py          MultiAgentPipeline; log
└── scripts/
    ├── acquisition/         01_fetch_github_issues.py
    │                        02_fetch_markdown_docs.py
    │                        03_fetch_pdfs.py
    └── evaluation/          generate_queries.py
                             check_queries.py
```

Reproduction of the full environment from a fresh clone requires four steps. First, dependencies are installed with `pip install -r requirements_agent.txt`; the agent file is a superset of the RAG requirements, so a single install covers both systems. Second, `.env.example` is copied to `.env` and three values are provided: `AI_REFINERY_BASE_URL`, `AI_REFINERY_API_KEY`, and `LLM_MODEL`. Third, the RAG index is built with `python -m rag.ingest`, which reads `data/corpus/`, chunks all records, computes embeddings, and writes the unified FAISS index to `rag/indexes/`; this step completes in approximately two minutes on M4. Fourth, the agent indexes are built with `python -m agent.ingest`, which performs the same pipeline filtered by source type for each of the three per-source indexes; this step completes in approximately one minute.

Three operational constraints govern the evaluation run. First, the AI Refinery LLM endpoint is hosted on Accenture's internal Kubernetes cluster and is unreachable without an active VPN connection; both pipelines fail with a connection timeout when the VPN is not engaged, as confirmed during development. Second, queries are submitted sequentially rather than in parallel — the endpoint does not publish rate limits, and sequential execution also eliminates concurrency artefacts from per-query wall-clock latency measurements, which are a primary evaluation metric. Third, token consumption across the 90-query evaluation substantially exceeded initial estimates: the RAG pipelines consumed 313,000–317,000 input tokens in total (reflecting one LLM call per query against up to five retrieved chunks, plus a short HyDE hypothesis call in the improved pipeline), while the single-agent pipeline consumed approximately 4.97 million logged input tokens — with an estimated true total of 8.2 million, since the 33 queries that raised `GraphRecursionError` recorded zero tokens due to a pipeline instrumentation gap corrected after the evaluation run. The multi-agent system consumed 919,000 input tokens across the same 90 queries. All systems remained within the available API budget.

5.2 RAG Pipeline Implementation

5.2.1 Chunking and Embedding

The ingestion script (`rag/ingest.py`) transforms raw corpus records into embedded chunk vectors in a single sequential pass: load all JSONL records, split each into overlapping windows, embed the resulting texts, and write a FAISS index alongside a companion chunk metadata file.

Chunking uses a word-count sliding window with a target size of 400 words and an overlap of 60 words, giving a step size of 340 words between window starts. The implementation splits on whitespace and joins the selected word slice back into a string — no tokeniser is involved. This word-based approach was chosen deliberately: it avoids a dependency on any specific tokeniser vocabulary, produces predictable chunk lengths across the three structurally different source types, and ensures that the same chunking code is shared without modification between the RAG and agent ingestion pipelines. Documents shorter than 400 words are emitted as a single chunk. Applied

to the 1,623-record corpus, the pipeline produces 4,840 chunks, each identified by a `chunk_id` of the form `<doc_id>__chunk<n>`.

The 400-word target balances two competing constraints. A chunk must be long enough to contain a complete technical thought, since a single sentence is too short for the cross-encoder reranker to assess relevance meaningfully, but short enough that the top-5 reranked chunks fit comfortably within the LLM context window alongside the system prompt. At a subword expansion ratio of approximately 1.3 tokens per word, five chunks of 400 words each contribute roughly 2,600 tokens of context, well within the limits of `gpt-oss-120b`.

Embeddings are computed with `BAAI/bge-base-en-v1.5`, a 768-dimensional bi-encoder that ranked among the top open-source retrieval models on the MTEB leaderboard at implementation time and requires no commercial licence. The model encodes all chunk texts with `normalize_embeddings=True`, which L2-normalises each vector before it is written to the FAISS index. Normalisation is required by the choice of `IndexFlatIP` as the index type: inner product on unit-norm vectors equals cosine similarity, so the index provides exact cosine search without approximation. Encoding is batched at 64 chunks per forward pass to control peak memory usage.

5.2.2 Retrieval and Reranking

Retrieval is implemented across four sequential stages in `rag/retrieve.py`: optional hypothesis generation, FAISS nearest-neighbour search, cross-encoder reranking, and Maximal Marginal Relevance diversification.

HyDE pre-retrieval. Before querying the FAISS index, the pipeline generates a hypothetical passage using the LLM. The `Generator.generate_hypothesis()` method in `rag/generate.py` issues a short generation call with the instruction "Write a short factual passage (2–3 sentences) that directly answers the question" and returns the completion text. The hypothesis is then embedded by `Retriever._embed_passage()`, which calls `model.encode()` without the BGE query prefix. Omitting the prefix is deliberate: the hypothesis is phrased in document register rather than question register and should therefore be embedded in the same distributional region as stored document chunks, not in the query subspace. The original query is retained separately for cross-encoder reranking, so relevance scoring in the next stage remains grounded in the actual question regardless of how the hypothesis was phrased.

FAISS search. When a hypothesis is available, `Retriever.retrieve()` passes the hypothesis embedding to `IndexFlatIP.search()` as the search vector; otherwise the query string is prefixed with the BGE retrieval instruction (*"Represent this sentence for searching relevant passages:"*) and its embedding is used instead. Because all stored vectors and all search vectors are L2-normalised, the inner product returned by `IndexFlatIP` equals cosine similarity exactly. The index returns the 20 highest-scoring candidates and their positions in the chunk metadata file. The 20-candidate pool provides a recall buffer that allows the cross-encoder to promote genuinely relevant passages that the bi-encoder placed outside the top 5.

Cross-encoder reranking. The cross-encoder (`cross-encoder/ms-marco-MiniLM-L-6-v2`) scores all 20 (original query, passage) pairs jointly. Joint scoring allows the model to attend to fine-grained token-level interactions between the query and the passage, giving substantially better relevance estimates than the inner-product score at the cost of 20 sequential forward passes. Candidates are sorted descending by cross-encoder score.

MMR diversification. After sorting by cross-encoder score, `Retriever._mmr()` selects the final 5 passages using Maximal Marginal Relevance. Cross-encoder scores are first normalised to $[0, 1]$ for commensurability with cosine similarity values. The stored L2-normalised embedding vector for each candidate is retrieved from the FAISS index via `IndexFlatIP.reconstruct()`. MMR then iterates: at each step it selects the candidate that maximises $\lambda \cdot \text{norm_score}(d_i) - (1 - \lambda) \cdot \max_{d_j \in S} \mathbf{v}_i^\top \mathbf{v}_j$, where S is the set of already-selected passages, $\mathbf{v}_i$ and $\mathbf{v}_j$ are stored embedding vectors, and $\lambda = 0.7$ (controlled by `MMR_LAMBDA` in `rag/config.py`). This prevents the final context window from being populated by five near-duplicate excerpts from the same document section. Both the FAISS score and the cross-encoder score are retained in each chunk's metadata and written to the query log.

The generator (`rag/generate.py`) constructs a user message from the top-5 chunks by numbering each passage and prepending its source type, title, and URL. A system prompt instructs the model to answer using only the provided context passages, to synthesise across all retrieved passages when the context contains information from multiple sources, and to decline explicitly when the context is insufficient. The LLM is called at temperature 0.0 for deterministic, reproducible output. Token counts are read from the API response's `usage` field and recorded in the log. The `RAGPipeline` class in `rag/pipeline.py` sequences all four stages, records wall-clock latency for

each stage — including `latency.hyde_s` for the hypothesis generation call — and appends a structured JSON record to a per-session log file.

5.2.3 Failure Modes and Resolutions

Four failure modes were encountered during development and resolved before the evaluation run.

Missing BGE query prefix. An early version of the retrieval code embedded query strings without the retrieval instruction prefix. Retrieval quality was measurably lower for several test queries — relevant passages appeared outside the top-20 candidate set — because the query vector fell in a region of the embedding space associated with general sentence similarity rather than passage retrieval. The fix was to apply the prefix unconditionally at query time, following the model card specification.

Extra body parameter rejected by AI Refinery. The initial generation call included `chat_template_kwargs` in the `extra_body` argument, a vLLM-specific extension used by some self-hosted deployments. The AI Refinery endpoint returned a validation error for this field. The resolution was to set `extra_body` to `None` when the configured base URL matches the AI Refinery hostname, a behaviour controlled by the `_skip_extra_body` flag in `rag/config.py`. The same flag is used by the agent configuration, keeping the workaround in one place.

VPN-gated endpoint. The AI Refinery endpoint is on Accenture's internal Kubernetes cluster and returns a connection timeout without VPN. This manifested as `httpx.ConnectTimeout` propagating up through the OpenAI client. The resolution is operational rather than code-level: the evaluation run must be initiated with an active VPN connection.

Unnormalised vectors in IndexFlatIP. A prototype built before normalisation was added to the ingest pipeline stored raw embedding vectors in the FAISS index. Inner-product scores for unnormalised vectors do not correspond to cosine similarity, so retrieval rankings were meaningless. The fix was to pass `normalize_embeddings=True` to `model.encode()` in both the ingest script and the query-time embedding call, ensuring that all vectors in the index and all query vectors lie on the unit hypersphere.

5.3 Agent Implementation

5.3.1 Agent Loop

The agent loop is constructed in `agent/agent.py` using `create_react_agent` from `langgraph.prebuilt`, which builds a compiled LangGraph `StateGraph` from an LLM and a list of tools. The resulting graph implements the ReAct pattern described by Yao et al. (2023): the model reasons about the query, emits a tool call, receives the retrieval result as an observation, and repeats until it produces a final answer without a tool call. In LangGraph 1.2.11 this loop is expressed through the model's native function-calling API rather than through text-parsed Thought/Action strings. Structurally, the pattern is unchanged, but the mechanism is more reliable because tool selection and argument extraction are handled by the model's structured output, not by fragile regex parsing of generated text.

Each iteration of the loop corresponds to two LangGraph supersteps: an agent node in which the LLM produces an `AIMessage` containing one or more `tool_calls`, and a tools node in which those calls are executed and the results are returned as `ToolMessage` objects appended to the state. The model receives the updated message history — including all prior tool observations — at the start of each subsequent agent node. The loop terminates when the model emits an `AIMessage` with no `tool_calls`, which is taken as the final answer. A recursion limit of 15 iterations is configured via `MAX_ITERATIONS` in `agent/config.py`. If this limit is reached, LangGraph raises a `GraphRecursionError`, which `AgentPipeline` catches and records as `finish_reason: "max_iterations"` in the query log.

One implementation obstacle arose from a breaking API change in LangChain 1.x. The `create_react_agent` function and `AgentExecutor` class had both been removed from the `langchain.agents` namespace and migrated to LangGraph, causing an `ImportError` at startup. The resolution was to import `create_react_agent` from `langgraph.prebuilt` and adapt the pipeline to LangGraph's message-based state, replacing the legacy `intermediate_steps` tuple list.

5.3.2 Tool Implementations

The three retrieval tools are implemented in `agent/tools.py`. Each tool is a Python function decorated with LangChain's `@tool` decorator, which registers the function's docstring as the description exposed to the model and wraps the return value as a `ToolMessage` observation. At runtime, each function delegates to a `SourceRetriever` instance bound to one of the three per-source FAISS indexes.

`SourceRetriever` encapsulates the same two-stage retrieval pipeline used in the RAG system: the query is embedded with the BGE prefix, the FAISS index returns 20 nearest-neighbour candidates by cosine similarity, and the cross-encoder reranks those candidates and returns the top 5. The embedding model, reranker, chunking parameters, and top-k settings are identical to those used in the RAG pipeline, loaded from the shared configuration values in `agent/config.py`. Because retrieval mechanics are identical, any quality difference observed in the evaluation reflects routing and synthesis behaviour, not per-chunk retrieval quality. The reranked chunks are formatted as a numbered list of passages — each with source type, title, URL, relevance score, and text — and returned as a plain string observation to the agent.

The markdown documentation index is built from a 400-record random sample of the full 1,283-record markdown corpus, specified by `SOURCE_LIMITS["markdown_doc"] = 400` in the agent configuration. This downsampling prevents the markdown source from dominating the per-source index by a factor of six over the other two sources and keeps agent index build time comparable to the RAG ingestion run. The downsampled subset is selected with a fixed random seed to ensure reproducibility.

Post-hoc reconstruction of which chunks were retrieved during a query is handled by the `ToolBundle` class. Each `SourceRetriever` appends the list of returned chunks to an internal `_call_history` list after every `search()` call. After the agent completes, `ToolBundle.collect_retrieved()` flushes these histories and matches each retriever call to the corresponding step in the agent trace using per-tool cursor indices. The result is a flat list of structured chunk records annotated with step number and tool name, written to the query log in the same schema as the RAG retrieved-chunks field. Because LangGraph exposes intermediate steps as message objects rather than `(AgentAction, observation)` tuples, a lightweight `_ToolCallProxy` adapter class in `agent/pipeline.py` bridges the two representations so that `collect_retrieved()` can process both without modification.

5.3.3 System Prompt and Routing Behaviour

The system prompt passed to `create_react_agent` establishes the agent's identity and constraints in five sentences: it names the role (enterprise knowledge assistant), enumerates the three available source categories (GitHub issues, technical documentation, CNCF whitepapers), instructs the model to use tools before answering, instructs it to search multiple sources when the question spans more than one, and constrains the final answer to retrieved evidence. No explicit routing rules are encoded in the prompt — the model is not told which tool to use for which query type.

Routing decisions emerge entirely from the match between query semantics and the natural-language descriptions registered in each tool's docstring, which specify appropriate use cases:

- `search_github_issues`: bug reports, feature requests, and error resolutions
- `search_markdown_docs`: conceptual explanations, how-to guides, and configuration references
- `search_pdf_whitepapers`: standards, best practices, and threat models

The model selects among these by attending to the query's vocabulary and intent, which is the routing mechanism authentic to the ReAct pattern (Yao et al., 2023). Encoding routing as tool descriptions rather than deterministic dispatch rules means the agent can handle query phrasings that do not exactly match any predefined category, at the cost of occasional mis-routing that is directly observable in the evaluation's routing accuracy metric.

The LLM is configured at temperature 0.0 throughout the ReAct loop, making both tool selection and answer generation deterministic for a given query. This is necessary for the evaluation: the routing accuracy metric compares the agent's `sources_used` log field against the ground-truth `source_type` labels in `data/eval/queries.jsonl`, and non-deterministic routing would make repeated runs produce different accuracy values from the same query set.

5.3.4 Runtime Failures and Resolutions

Three failure modes were identified during the single-agent evaluation run and resolved through targeted fixes to `agent/pipeline.py` and `agent/config.py`.

Reasoning-model content format. `gpt-oss-120b` returns `msg.content` as a typed block list — for example, `[{"type": "text", "text": "..."}]` — rather than a plain string. The original answer extraction code applied an `isinstance(content, str)` guard and silently discarded non-string values, causing four queries to record an empty answer despite successful ReAct loop completion. The fix joins all blocks of type `"text"` or `"thinking"` into a single string before the guard check.

Token capture on recursion error. The `get_openai_callback()` context manager accumulates token counts across all LLM calls made during `app.invoke()` and its counters are populated by the time the context exits — even when `app.invoke()` raises a `GraphRecursionError`. The original pipeline code read `cb.prompt_tokens` inside the `try` block after the `invoke` call, where it was skipped by exception propagation. The fix initialises `cb = None` before the `try` block and reads token counts in the `except GraphRecursionError` branch. Token counts for the 33 already-logged failure records remain at zero in the definitive evaluation log and are noted as understated in Chapter 6.

Unanswerable queries and the iteration ceiling. The original system prompt contained no instruction for queries whose answers are absent from the corpus. Without an explicit decline path, the agent searched all three indexes exhaustively on every unanswerable query and reached the iteration ceiling in all 15 such cases. A single sentence was added to the system prompt: "If after searching the available sources you cannot find relevant information, state clearly that the answer is not available in the knowledge base — do not continue searching." The maximum completion budget was also raised from 1,024 to 2,048 tokens to ensure that `gpt-oss-120b`'s internal reasoning tokens do not exhaust the response budget before the visible answer is written. Both fixes were applied after the evaluation run and informed the design of the multi-agent architecture described in the following subsection.

5.3.5 Multi-Agent Coordination Architecture

The single-agent evaluation produced a 41 % failure rate and a mean latency of 16.3 seconds per query, primarily because the sequential ReAct loop accumulated context across up to 9 tool calls and hit the iteration ceiling on queries where the model searched exhaustively rather than declining. These failure modes motivated a second agent architecture, implemented in `multi_agent/`, that decouples retrieval from reasoning and eliminates the ReAct loop entirely.

The multi-agent graph is a two-node `StateGraph` defined in `multi_agent/graph.py`. A `retrieve_all_sources` node iterates over the three source types sequentially, calling the `SourceRetriever._retrieve()` method from `agent/tools.py` for each and accumulating the returned chunk lists. An initial design used LangGraph's `Send()` API to dispatch these retrieval calls as parallel fan-out branches; this was reverted because Apple Silicon MPS does not support concurrent `SentenceTransformer` inference across threads and produced a segfault during execution. Sequential iteration over three FAISS indexes takes approximately 300 ms and represents a negligible fraction of per-query latency, so the reversion had no measurable effect on throughput. A `synthesize` node then receives the pooled passages — at most 15 chunks, 5 per source — and issues a single LLM call to produce the final answer.

The `MultiAgentState` TypedDict carries `query`, `source_results`, `answer`, and `finish_reason`; there is no iteration limit and no `GraphRecursionError` risk. The synthesis prompt instructs the model to integrate evidence from all three source types and to decline explicitly when none of the retrieved passages addresses the query. The LLM call is wrapped in `get_openai_callback()` identically to the single-agent pipeline, so token counts are captured correctly for all 90 queries. The `MultiAgentPipeline` class in `multi_agent/pipeline.py` exposes the same `query(question)` interface as `AgentPipeline` and writes log records in a schema compatible with the evaluation scripts.

The multi-agent implementation reuses the per-source FAISS indexes built by `agent/ingest.py` and the `SourceRetriever` class from `agent/tools.py`, adding no new ingestion code. The effective structural difference from the single-agent system is the elimination of the decision loop: the multi-agent always queries all three

sources and makes exactly one LLM call, giving it a bounded, fixed-size context per query regardless of query type.

5.4 Logging and Instrumentation

Both pipelines write structured JSON logs to enable post-hoc evaluation without re-running queries. Every query produces one JSON record appended to a per-session log file; evaluation scripts consume these files directly, without any intermediate transformation. The RAG and agent logs share an identical set of outer fields — making cross-system comparison a straightforward field-by-field join on `query` — and the agent log extends this shared schema with three additional fields that have no RAG equivalent. Table 5.2 lists the complete field set for both systems.

Latency is measured with `time.perf_counter()`, which provides sub-millisecond wall-clock resolution and is not affected by system clock adjustments. In the RAG pipeline, timing is split across three instrumented blocks: `latency.hyde_s` wraps the `Generator.generate_hypothesis()` call and is recorded as 0.0 when `USE_HYDE` is disabled; `latency.retrieve_s` wraps the `Retriever.retrieve()` call, covering FAISS search, cross-encoder reranking, and MMR diversification; and `latency.generate_s` wraps the `Generator.generate()` call covering the LLM API round-trip only. The agent and multi-agent pipelines record only `latency.total_s`, wrapping the entire `app.invoke()` call. A per-stage breakdown is not recorded for either agent system because the number of LLM calls and retrieval operations varies per query in the single-agent case, while the multi-agent always performs the same two-node sequence; in neither case would stage-level timing add evaluation value. In all three systems, the timer is stopped before the log write, so file I/O is excluded from all latency figures.

Token consumption is captured by two different mechanisms that produce the same `tokens` field structure. For the RAG pipeline, prompt and completion token counts are extracted from the `usage` object that the OpenAI API returns alongside every chat completion. For the agent pipeline, a callback context manager is used instead, because the multi-step loop makes per-response token access impractical.

The `get_openai_callback()` function from `langchain_community` wraps the `app.invoke()` call and intercepts every LLM invocation the LangGraph StateGraph

makes internally, accumulating prompt and completion tokens across all steps of the ReAct loop. Without it, token totals would require summing counts from each `AIMessage` individually, which the LangGraph API does not directly expose. If `langchain_community` is unavailable, the agent pipeline falls back to reporting zero for both token fields.

The agent trace is constructed after `app.invoke()` returns by scanning the final message list in order. Each `AIMessage` that contains `tool_calls` is processed to create a partial trace entry — recording the step number, the tool name, and the query argument — stored in a pending dictionary keyed by `tool_call_id`. Each subsequent `ToolMessage` is matched to its pending entry by `tool_call_id` and the observation text, truncated to 3 000 characters, is attached to complete the entry. Completed entries are appended to `agent_trace` in execution order. The `sources_used` field is derived as the sorted unique set of tool names from the completed trace, and `tool_calls` is set to the length of the trace list. Truncating observations at 3 000 characters bounds log file size while retaining enough context for manual inspection of routing decisions.

Each pipeline instance writes to a single JSONL file named with a UTC timestamp at initialisation time — for example, `rag/logs/rag_20260812T193235.jsonl`. Records are appended one per line in the format `json.dumps(result) + "\n"`, which produces a valid JSONL file that evaluation scripts can process with a single line-by-line read. Using a per-session file rather than a single accumulating log prevents query records from different runs from interleaving and allows partial evaluation runs to be identified and re-run by filename timestamp.

Table 5.1: Project dependencies with confirmed installed versions (verified 2026-08-17).

Library	Version	Role
Inference stack		
`openai`	2.54.0	RAG LLM client (OpenAI-compatible REST API); downgraded from 3.0.0 by ragas dependency resolution
`sentence-transformers`	5.7.0	BGE-base-en-v1.5 embeddings; CrossEncoder reranker
`transformers`	5.15.0	HuggingFace model loading (used by sentence-transformers)
`faiss-cpu`	1.15.0	`IndexFlatIP` nearest-neighbour search
`numpy`	2.5.2	L2 normalisation; cosine similarity
Agent framework		
`langgraph`	1.2.11	Agent loop (`create_react_agent`, StateGraph)
`langchain`	1.3.15	Core abstractions
`langchain-openai`	1.5.1	`ChatOpenAI` wrapper for agent LLM calls
`langchain-core`	1.5.5	`@tool` decorator; `AIMessage`; `ToolMessage`
`langchain-community`	0.4.2	`get_openai_callback()` for token counting
Utilities		
`python-dotenv`	1.2.2	Loads `.env` into `os.environ` at startup
`tqdm`	4.70.0	Ingestion progress bars
`requests`	2.34.2	GitHub REST API calls (acquisition scripts)
`pdfplumber`	0.11.10	PDF text extraction (acquisition scripts)
Evaluation		
`ragas`	0.4.3	Context relevance, faithfulness, answer relevance
`bert-score`	0.3.13	BERTScore F1 (microsoft/deberta-xlarge-mnli)
`rouge-score`	0.1.2	ROUGE-L lexical overlap
`datasets`	5.0.1	Required by ragas for Dataset input format

Table 5.2: Log record schema. Shared fields are produced identically by both pipelines; agent-only fields have no equivalent in the RAG log.

Field	Type	Description
Shared fields (RAG and agent)		
`timestamp`	string	UTC ISO-8601 timestamp, set after query completes
`system`	string	`"rag"`, `"agent"`, or `"multi_agent"`
`query`	string	The input question
`answer`	string	The generated answer
`finish_reason`	string	`"stop"` or `"max_iterations"`
`retrieved`	array	Chunk metadata: chunk_id, source_type, doc_id, title, faiss_score, rerank_score
`latency.total_s`	float	Wall-clock time, query start to answer complete
`tokens.input`	int	Prompt tokens consumed
`tokens.output`	int	Completion tokens generated
`tokens.total`	int	Sum of input and output tokens
RAG-only additional fields		
`latency.hyde_s`	float	HyDE hypothesis generation time (0.0 when disabled)
`latency.retrieve_s`	float	FAISS search, cross-encoder reranking, and MMR time
`latency.generate_s`	float	LLM generation call time
`hyde_hypothesis`	string	Generated hypothesis used for FAISS search (null when disabled)
Agent-only fields		
`agent_trace`	array	One entry per tool call: step, tool name, tool input, observation
`tool_calls`	int	Number of completed tool calls (`len(agent_trace)`)
`sources_used`	array	Sorted unique tool names from the trace

6 Evaluation Results

6.1 Retrieval and Generation Quality

Table 6.1 reports context precision, answer relevancy, BERTScore F1, and ROUGE-L for factual and synthesis queries across all four systems. The four systems are: the original RAG pipeline (no HyDE or MMR), the single-agent ReAct system, the multi-agent coordinator, and the improved RAG pipeline (HyDE, MMR, and synthesis prompt). Ambiguous and unanswerable queries are discussed in Section 6.4 because the absence of well-defined ground-truth strings makes their metric values unreliable for direct cross-system comparison.

Table 6.1: Quality metrics by system and query type. BERTScore uses `microsoft/deberta-xlarge-mnli` with baseline rescaling; values above zero indicate performance above the baseline model. † Multi-agent factual context precision is unavailable: RAGAS timed out on that subset.

System	Query type	Context prec.	Ans. relevancy	BERTScore F1	ROUGE-L
Original RAG	Factual	0.546	0.792	0.217	0.346
Original RAG	Synthesis	0.730	0.679	0.128	0.176
Single Agent	Factual	0.338	0.663	−0.260	0.151
Single Agent	Synthesis	0.315	0.547	−0.355	0.080
Multi-Agent	Factual	—†	0.818	0.034	0.187
Multi-Agent	Synthesis	0.308	0.770	0.060	0.117
Improved RAG	Factual	0.518	0.804	0.232	0.349
Improved RAG	Synthesis	0.708	0.742	0.133	0.165

The multi-agent system achieves the highest answer relevancy on both factual queries (0.818) and synthesis queries (0.770), exceeding all other systems on the metric most directly associated with response usefulness. The improved RAG pipeline ranks second

on both query types, at 0.804 on factual and 0.742 on synthesis, with the synthesis improvement of 0.063 points over original RAG representing the primary benefit of HyDE and MMR: better evidence selection on multi-document queries. The original RAG achieves higher factual answer relevancy than the single agent (0.792 versus 0.663), and the gap is present but smaller on synthesis queries (0.679 versus 0.547).

Context precision presents a more nuanced pattern. Original RAG achieves the highest context precision on both factual queries (0.546) and synthesis queries (0.730), while improved RAG is marginally lower on both (factual 0.518, synthesis 0.708). This counter-intuitive result is consistent with MMR's function: by selecting contextually distinct passages over the most similar ones, MMR may introduce some passages that are topically related but not strictly necessary for the query, depressing the RAGAS precision score while enriching the evidence available for synthesis. The single agent's low context precision (0.315–0.338) is substantially explained by the 41 % empty-answer failure rate, which contributes zero precision scores across those 37 records.

BERTScore and ROUGE-L follow the same directional pattern as answer relevancy. Both RAG variants score above the DeBERTa rescaling baseline on factual and synthesis queries; the multi-agent is marginally above baseline (0.034–0.060); the single agent scores well below baseline across both query types due to empty answers. The lexical and semantic similarity metrics are insensitive to the HyDE and MMR improvements: original RAG ROUGE-L factual is 0.346 and improved RAG is 0.349 — a negligible difference. This confirms that the retrieval improvements change the relevance and coherence of answers without substantially shifting their surface form. The multi-agent's relatively low BERTScore and ROUGE-L despite leading on answer relevancy reflects a structural property: the synthesiser receives fifteen chunks from three sources and produces longer, integrative answers that diverge in surface form from the concise ground-truth strings used as reference texts.

Table 6.2 reports faithfulness where the RAGAS evaluation completed within the available time budget. The single agent's answers could not be evaluated on any query type; the multi-agent's faithfulness is available only for unanswerable queries; both systems produce longer reasoning-enriched responses that exhausted the RAGAS LLM judge's per-query time allocation.

Among the measurable rows, several findings emerge. Original RAG achieves its highest faithfulness on factual queries (0.853), reflecting that single-chunk answers are concise and fully grounded. Improved RAG scores lower on factual queries (0.799),

Table 6.2: Faithfulness by system and query type. Unavailability is due to RAGAS LLM judge timeout on long responses.

System	Query type	Faithfulness
Original RAG	Factual	0.853
Original RAG	Ambiguous	0.707
Original RAG	Unanswerable	0.717
Original RAG	Synthesis	n/a (timeout)
Improved RAG	Factual	0.799
Improved RAG	Ambiguous	0.819
Improved RAG	Unanswerable	0.778
Improved RAG	Synthesis	n/a (timeout)
Multi-Agent	Unanswerable	0.839
Multi-Agent	Other types	n/a (timeout)
Single Agent	All types	n/a (timeout)

consistent with the synthesis prompt's instruction to draw from multiple passages — answers citing several chunks are somewhat more likely to include a claim that the RAGAS judge cannot trace to a single retrieved passage. On ambiguous and unanswerable queries, improved RAG faithfulness exceeds original RAG's scores (ambiguous: 0.819 versus 0.707; unanswerable: 0.778 versus 0.717), suggesting that the updated decline instruction in the synthesis prompt produces better-grounded refusals. The multi-agent achieves the highest unanswerable faithfulness of any system (0.839), because its refusals are grounded in three sets of retrieved evidence that all fail to address the query, providing a stronger signal for decline than a single-source retrieval failure.

Table 6.3 reports source routing accuracy for both agent-based systems. Multi-agent routing is 100 % by architectural design: the system always queries all three sources per query regardless of query characteristics. Single-agent routing achieved 73.3 % overall, with synthesis queries showing the lowest accuracy (70.0 %) and ambiguous queries the highest (80.0 %).

Table 6.3: Source routing accuracy by system and query type. Unanswerable queries are excluded because no source type is expected. Multi-agent routing is 100 % by architectural design.

Query type	Queries	Routing accuracy	
		Single Agent	Multi-Agent
Factual	30	73.3 % (22/30)	100 % (30/30)
Synthesis	30	70.0 % (21/30)	100 % (30/30)
Ambiguous	15	80.0 % (12/15)	100 % (15/15)
Overall	**75**	**73.3 % (55/75)**	**100 % (75/75)**

6.2 Latency and Cost

Table 6.4 reports response latency statistics across all 90 queries for each system, together with per-system failure rates. All measurements are wall-clock times captured with `time.perf_counter()` on the same M4 workstation over the same AI Refinery API endpoint; hardware and network conditions are therefore identical across all four systems.

Table 6.4: Response latency (seconds) across all 90 queries per system. Failed queries (empty answers) are included in latency statistics — they consumed the full iteration budget before failing.

System	Mean	Median	Std dev	Min	Max	Failure
Original RAG	2.506	2.032	1.174	1.047	5.837	0 %
Single Agent	16.333	17.376	8.174	2.160	29.140	41 %
Multi-Agent	4.302	3.138	2.487	1.699	10.505	0 %
Improved RAG	4.279	3.687	1.744	2.061	9.881	0 %

The multi-agent and improved RAG systems occupy a middle ground between the original RAG pipeline and the single agent. Multi-agent mean latency of 4.302 seconds is 1.7 times original RAG's 2.506 seconds; improved RAG mean latency of 4.279 seconds is essentially identical to the multi-agent, despite a fundamentally different architecture — the HyDE hypothesis call adds approximately 1.8 seconds per query, replacing the single-agent's iteration overhead with a single bounded extra LLM call. The single agent's mean latency of 16.333 seconds is 6.5 times original RAG, with a standard

deviation of 8.174 seconds indicating per-query variability that would make latency budgeting impractical in production. The multi-agent's standard deviation of 2.487 seconds is lower than the single agent's by a factor of 3.3, reflecting the absence of non-deterministic routing decisions.

Table 6.5 reveals the sharpest divergence on unanswerable queries. Original RAG handles unanswerable queries in 1.518 seconds on average — its fastest category — because irrelevant retrieval surfaces quickly and the LLM produces a short declining response. The multi-agent handles the same queries in 2.141 seconds, also its fastest category: with all three source sets returning poor evidence, the synthesiser recognises the absence of relevant information and declines concisely. The single agent takes 22.850 seconds on average for unanswerable queries — fifteen times longer than original RAG and the slowest of any system on any query type — because without the explicit decline instruction present in the corrected system prompt, the agent searched exhaustively through the full iteration ceiling on all 15 unanswerable queries. Improved RAG handles unanswerable queries in 2.978 seconds; the HyDE call is performed regardless of query type, adding a fixed overhead over original RAG.

Table 6.5: Mean latency (seconds) by query type and system.

Query type	**Orig. RAG**	**Single Agent**	**Multi-Agent**	**Imp. RAG**
Factual	2.065	12.563	3.198	3.742
Synthesis	3.618	17.844	6.902	5.801
Ambiguous	2.153	14.335	3.469	3.608
Unanswerable	1.518	22.850	2.141	2.978

Table 6.6 reports total token consumption and estimated cost for the full 90-query evaluation run. Cost estimates use GPT-4o proxy pricing of $2.50 per million input tokens and $10.00 per million output tokens, applied uniformly; AI Refinery pricing is not published externally. The single-agent input token count is understated: 33 queries that raised `GraphRecursionError` recorded zero tokens due to a pipeline instrumentation gap corrected after the evaluation run; the estimated true agent input total is approximately 8.2 million tokens and the estimated true cost approximately $21.

The cost comparison produces a clear ranking. Original RAG and improved RAG are essentially equivalent at $1.18 and $1.20 respectively: HyDE adds only $0.02 per 90

Table 6.6: Token consumption and estimated cost for 90 queries. Cost estimates use GPT-4o proxy pricing ($2.50/M input, $10.00/M output). † Single-agent input is understated; estimated true total is approximately 8.2M tokens ($21).

System	Input tokens	Output tokens	Total tokens	Est. cost
Original RAG	316,712	38,495	355,207	$1.18
Single Agent	4,966,751†	82,184	5,048,935†	$13.24†
Multi-Agent	919,436	60,909	980,345	$2.91
Improved RAG	312,843	42,114	354,957	$1.20

queries because the hypothesis call uses a short prompt and produces 2–3 sentences. Multi-agent costs $2.91, 2.5 times original RAG, because it always performs three retrieval passes and one synthesis call with a larger pooled context. Single agent costs $13.24 as logged, and an estimated $21 at true token consumption, placing it at 11–18 times original RAG cost. Projected to 1,000 queries, original and improved RAG cost approximately $13–$14, multi-agent approximately $32, and single agent approximately $147–$233. The improved RAG's HyDE hypothesis call adds one extra LLM invocation per query but contributes negligibly to cost because hypothesis prompts are short and generation is capped at 512 tokens.

6.3 Implementation Complexity and Maintainability

The complexity rubric defined in Section 4.3.5 assigns scores from 1 (highest complexity) to 5 (lowest complexity) across five criteria. Both RAG variants share the same codebase; the improved pipeline enables two configuration flags (`USE_HYDE = True`, `USE_MMR = True`) and extends three files with 122 additional lines, increasing RAG's total from 377 to 499 LoC. The scores below reflect the improved RAG configuration, which is the version evaluated. The multi-agent system is evaluated as a distinct architecture. Table 6.7 presents scores for all three architectures.

The multi-agent architecture occupies a consistent intermediate position between RAG and the single agent on every criterion; Figure 6.1 shows this pattern across all five criteria simultaneously. The most pronounced difference between multi-agent and single-agent scores is debuggability: replacing the non-deterministic ReAct loop with a fixed two-node graph eliminates the routing failure mode and the max-iterations

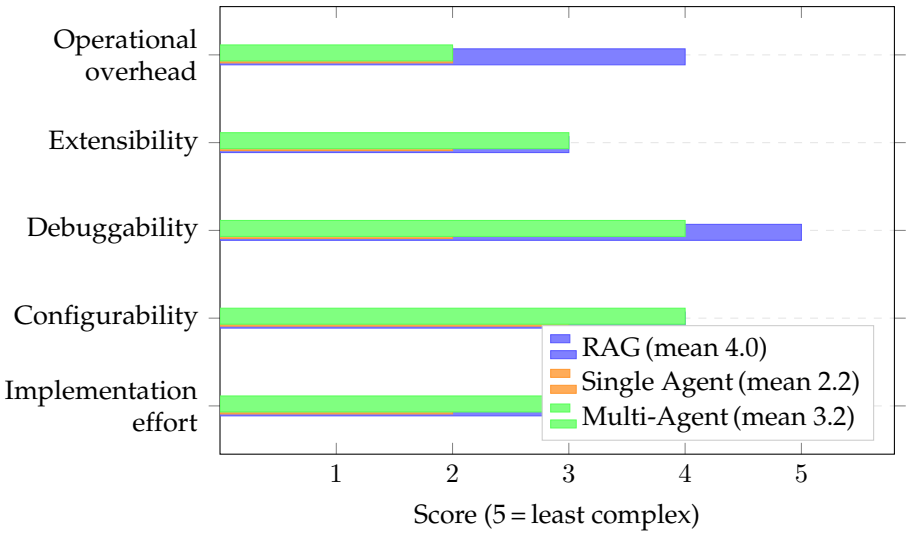


Figure 6.1: Per-criterion complexity scores from Table 6.7. Higher score indicates lower complexity or simpler maintainability. RAG leads on four of five criteria; the multi-agent occupies the intermediate position on all five; the single agent scores lowest on four of five.

boundary as distinct diagnostic categories. A wrong multi-agent answer is attributable either to poor retrieval from one or more sources — diagnosable by inspecting FAISS and cross-encoder scores in the log — or to synthesis quality, diagnosable by inspecting the pooled context passed to the LLM. The single agent encountered three failure modes during evaluation not present in either RAG variant: routing errors, hallucinated tool names, and the max-iterations ceiling — each requiring different diagnostic procedures.

The extensibility scores are equal for RAG and multi-agent (both 3), but the steps involved differ in kind. RAG requires extending the corpus and rebuilding a single unified index; the multi-agent requires adding a new `SourceRetriever` instance without touching the synthesis logic. The single agent's extensibility score of 2 reflects the additional step of re-evaluating routing behaviour after adding a tool, since the new docstring description may alter how the model distributes queries across existing tools.

6.4 Robustness Observations

Negative Rejection

The negative rejection dimension from Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang (2023) concerns the system's ability to decline answering questions whose answer does not exist in the corpus. The 15 unanswerable queries produced distinct patterns across the four systems. Original RAG returned nominally relevant-looking responses on all 15 queries — its answer relevancy averaged 0.0 for that category, indicating the RAGAS judge found these responses irrelevant despite them being grounded in retrieved context. The context precision of 0.033 confirms that retrieved passages were almost entirely irrelevant, yet the model did not consistently recognise this as a signal to decline. Improved RAG showed structurally similar behaviour but with a higher unanswerable faithfulness (0.778 versus 0.717 for original RAG), attributable to the updated synthesis prompt's explicit decline instruction.

The multi-agent system produced the cleanest unanswerable behaviour of any system: its unanswerable latency of 2.141 seconds is the lowest of any system on any query type, and its unanswerable faithfulness of 0.839 is the highest across all measurable rows in Table 6.2. Receiving uniformly poor evidence from all three source searches simultaneously appears to produce a stronger and more consistent signal for the synthesiser to decline than a single-source retrieval failure. The single agent handled unanswerable queries worst: all 15 queries reached the iteration ceiling, producing empty answers after 22.850 seconds of wasted search effort per query on average.

Single-Agent Empty-Answer Failures

The most significant robustness finding across all four systems is that 37 of 90 single-agent queries (41 %) returned an empty answer string. The failure mode split into two distinct subtypes. The dominant subtype (33 queries) was `GraphRecursionError` failures in which the agent exhausted the 15-iteration ceiling; these were concentrated in unanswerable queries (15/15, 100 %), synthesis queries (11/30, 37 %), and factual queries (8/30, 27 %). The secondary subtype (4 queries) returned `finish_reason: "stop"` with empty content, caused by the reasoning model's internal chain-of-thought consuming the completion budget before the visible answer was written. Empty

answers contribute a BERTScore of -1.06 (the rescaled floor value) and a ROUGE-L of 0 to the system means in Table 6.1. Neither the multi-agent nor either RAG variant produced any empty answers across all 90 queries.

Noise Robustness and Context Precision

Original RAG achieved the highest context precision on synthesis queries (0.730), reflecting the unified index's ability to surface relevant chunks from multiple documents in a single FAISS pass without a routing decision. The improved RAG is marginally lower (0.708) for the reason described in Section 6.1: MMR diversification selects contextually distinct passages over the most similar ones, which may include topically related but not strictly necessary passages and consequently depress the RAGAS precision score. The multi-agent achieves synthesis context precision of 0.308 — lower than both RAG variants — for two compounding reasons: the per-source indexes are downsampled (the markdown index covers 400 of 1,283 records), and the fixed 5-chunks-per-source retrieval ceiling means the synthesiser always receives 15 chunks regardless of relevance, diluting precision when some sources contain no relevant material for a given query.

Evaluation Metric Limitations

Three evaluation metric limitations affected the completeness of results. First, multi-agent factual context precision is unavailable because RAGAS timed out on that subset; the omission is noted in Table 6.1 and does not affect the primary answer relevancy comparison. Second, faithfulness measurement was unavailable for the single agent on all query types and for the multi-agent on all types except unanswerable, as documented in Table 6.2; this prevents any conclusion about whether the agent-based systems produce more or less hallucinated content than the RAG variants. Third, BERTScore values for ambiguous and unanswerable queries reach the rescaled floor of -1.06 for all systems, because those query types either lack ground-truth strings or have ground-truth strings whose reference interpretation does not align with the system's response scope; these values are excluded from Table 6.1 and are not used in cross-system comparisons. All three limitations are noted as constraints on the scope of the findings in Section 7.3.

Table 6.7: Complexity and maintainability rubric scores (1 = most complex, 5 = least complex). Improved RAG shares the RAG codebase with two additional configuration flags and receives identical scores.

Criterion	Justification	RAG	Agent	M-Agt
Implementation effort	**RAG:** 499 LoC; five files, no orchestration framework **Agent:** 600 LoC; adds tool layer, multi-step logging, LangGraph wiring **M-Agt:** 961 LoC total; adds graph, synthesis node, fan-in logic	4	2	3
Configurability	**RAG:** 12 tunable parameters **Agent:** 16; adds per-tool top-k, iteration limit, routing prompt **M-Agt:** 10; no iteration limit; shorter synthesis prompt	4	3	4
Debuggability	**RAG:** Linear pipeline; fault isolates to retrieval score or LLM call **Agent:** Non-deterministic routing; max-iterations failures require full trace reconstruction **M-Agt:** Fixed two-node graph; fault isolates to retrieval or synthesis stage	5	2	4
Extensibility	**RAG:** Extend corpus JSONL + rebuild one index (2 steps) **Agent:** New index + tool + prompt + routing re-evaluation (5 steps) **M-Agt:** Add SourceRetriever + update retrieve loop (3 steps)	3	2	3
Operational overhead	**RAG:** One LLM API; one FAISS index **Agent:** Same API + 3 FAISS indexes + LangGraph + LangChain tooling **M-Agt:** Same as single agent; equivalent dependency surface	4	2	2
Unweighted mean		**4.0**	**2.2**	**3.2**

7 Discussion

7.1 Answering the Research Questions

RQ1: How does a classical RAG pipeline compare to an agent-based approach in terms of latency, cost, and retrieval performance in enterprise environments?

The direct comparison between the original RAG pipeline and the single-agent ReAct system confirms RAG's advantage across all three dimensions. On retrieval performance, RAG achieves a context precision of 0.546 on factual queries and 0.730 on synthesis queries, against the agent's 0.338 and 0.315; it also produces higher answer relevancy on both query types. On latency, the RAG pipeline completes queries in a mean of 2.506 seconds against the agent's 16.333 seconds, a factor of 6.5. On token cost, the agent consumes 15.7 times as many input tokens as RAG over the 90-query evaluation, yielding an estimated cost ratio of 11.2 to 1. These are not marginal differences: the agent's disadvantage on each dimension is large enough to dominate the deployment decision wherever latency budgets or token costs are constrained. The agent's 41 % query failure rate further compounds the quality deficit.

The four-system evaluation qualifies this finding in two important ways. First, the multi-agent coordinator architecture eliminates the single-agent's primary failure modes, namely the iteration ceiling and context accumulation, and reduces the cost ratio to 2.5× RAG while achieving higher answer relevancy on both query types (factual: 0.818 versus 0.792; synthesis: 0.770 versus 0.679). This demonstrates that the original comparison's verdict against agent-based retrieval is specific to the single-agent ReAct design, not to the agentic paradigm in general. Second, improving the RAG pipeline with HyDE and MMR raises synthesis answer relevancy from 0.679 to 0.742 at effectively the same cost ($1.18 versus $1.20), establishing that significant quality gains are available without architectural change.

RQ2: To what extent does an agent-based system improve source selection and answer quality compared to RAG?

This question is examined across two agent architectures: the single-agent ReAct system established in the original experimental design, and the multi-agent coordinator developed in response to the failure modes identified during single-agent evaluation. The answer depends on which agent architecture is evaluated. The single-agent ReAct system does not improve answer quality relative to original RAG on any measured metric, despite providing a genuine routing capability through which 73.3 % of queries are directed to the correct source type through natural-language tool descriptions alone. The routing accuracy is a structural property with no equivalent in RAG, but in this evaluation it does not translate to better answers: the multi-step context accumulation and the 41 % failure rate offset any quality gain from selective source targeting.

The multi-agent coordinator, by contrast, improves answer relevancy over both RAG variants on every applicable query type. Factual answer relevancy reaches 0.818, the highest of any system, against original RAG's 0.792 and improved RAG's 0.804. Synthesis answer relevancy reaches 0.770, against original RAG's 0.679 and improved RAG's 0.742. The critical architectural difference is the elimination of the decision cycle: the multi-agent always queries all three sources and synthesises from a bounded 15-chunk context, ensuring that the synthesiser has access to the best available evidence from each domain without the noise introduced by iterative context accumulation. Taken together, these findings suggest that the agent's structural advantage in multi-source queries, which derives from access to evidence from multiple independent retrieval systems, materialises only when the iteration-ceiling failure mode is removed. Source selection itself is not the operative factor, since the multi-agent's 100 % routing coverage is a property of its design rather than a reasoning capability.

RQ3: What are the differences in implementation complexity and maintainability between both approaches?

The rubric assessment in Section 6.3 assigns RAG an unweighted mean score of 4.0 out of 5, the single agent 2.2, and the multi-agent coordinator 3.2. The multi-agent occupies a consistent intermediate position across all five criteria. The most substantial gap between the single agent and both other architectures is debuggability: the non-deterministic routing, multi-step context accumulation, and max-iterations boundary of the ReAct cycle each require different diagnostic procedures, while both RAG variants and the multi-agent reduce to inspecting stage-level scores in a fixed execution graph.

In code volume, the improved RAG implementation stands at 499 lines, the single agent at 600 lines, and the multi-agent at 361 lines of new code over the 600-line agent infrastructure (961 lines total required).

The extensibility comparison reveals a meaningful difference in the kind of work required to add a new data source. RAG requires two steps, namely extending the corpus and rebuilding a single unified index, with no change to any code path. The multi-agent requires three steps, adding a `SourceRetriever` instance and updating the retrieval cycle, also without structural change. The single agent requires five steps, including writing and registering a new `@tool` function and re-evaluating whether its docstring description produces correct routing behaviour, a change that may alter routing accuracy for existing tools and cannot be validated without re-running the evaluation.

RQ4: Under which conditions is each approach preferable in practice?

The four-system evaluation yields a nuanced practical recommendation that depends on deployment priorities.

The *original RAG pipeline* is the appropriate choice when cost and latency are primary constraints, the query mix consists predominantly of factual lookups against a single knowledge domain, and the engineering team has limited experience with agentic frameworks. At $1.18 per 90 queries and a mean latency of 2.506 seconds, it is the most economical and predictable system evaluated.

The *improved RAG pipeline* (HyDE and MMR) is the right upgrade path when synthesis quality matters and the cost and infrastructure are to remain unchanged. Adding HyDE and MMR raises synthesis answer relevancy by 9 % at a cost increase of $0.02 per 90 queries and a latency increase of 1.7×. No new infrastructure, no new dependencies, and no change to the deployment architecture are required.

The *single-agent ReAct system* as implemented offers no advantage over any alternative evaluated here and should not be deployed without a hard cap on tool calls, a reliable decline path for unanswerable queries, and explicit confirmation that the LLM model does not produce a reasoning-heavy response style that saturates the context budget.

The *multi-agent coordinator* is the preferred choice when maximum answer quality across all query types is the priority. It delivers the highest answer relevancy on factual and synthesis queries, handles unanswerable queries with the highest faithfulness of any

system (0.839), and produces no empty-answer failures. Its cost of $2.91 per 90 queries is 2.5× original RAG, a manageable premium given that the single-agent alternative costs 11× RAG for worse results. Its debuggability score of 4 and its simpler configuration make it substantially more maintainable than the single-agent system it supersedes.

7.2 Trade-Off Analysis

Table 7.1 summarises the comparative position of all four systems across the five evaluation dimensions. Ratings are qualitative assessments anchored to the specific numbers reported in Chapter 6.

Table 7.1: System-level trade-off comparison across five evaluation dimensions. CP = context precision; AR = answer relevancy.

Dimension	Orig. RAG	Single Agent	Multi-Agent	Imp. RAG
Retrieval quality	High (CP: 0.55–0.73)	Low (CP: 0.31–0.34; 41 % failure)	Moderate† (AR: 0.82 factual)	High (CP: 0.52–0.71)
Answer quality	High (AR: 0.68–0.79)	Low (AR: 0.55–0.66; 41 % failure)	Highest (AR: 0.77–0.82)	High (AR: 0.74–0.80)
Latency	Low (2.5s mean)	High (16.3s; stdev 8.2s)	Moderate (4.3s mean)	Moderate (4.3s mean)
Token cost	Low ($1.18)	High ($13.24; 11×)	Moderate ($2.91; 2.5×)	Low ($1.20)
Impl. complexity	Low (499 LoC; mean 4.0)	High (600 LoC; mean 2.2)	Moderate (961 LoC; mean 3.2)	Low (499 LoC; mean 4.0)

† Multi-agent factual context precision unavailable (RAGAS timeout).

The four-system comparison reveals a more differentiated landscape than the original two-system evaluation. The single agent occupies the worst position on every dimension simultaneously, which makes it uniquely unsuitable as a production architecture in its evaluated form. The multi-agent and improved RAG systems each solve one of the two original failure modes: the multi-agent eliminates the reliability and quality failures of the single agent; the improved RAG eliminates the synthesis quality gap of the original pipeline. Both arrive at similar latency of approximately 4.3 seconds mean

by structurally different routes, the multi-agent through parallel source access and the improved RAG through a single extra hypothesis call.

The remaining trade-off between the multi-agent and improved RAG is concentrated in two dimensions: answer quality and cost. Multi-agent leads on answer relevancy on both query types (factual: 0.818 versus 0.804; synthesis: 0.770 versus 0.742) and on unanswerable faithfulness (0.839 versus 0.778), at a cost of $2.91 against improved RAG's $1.20. Whether this 2.4× cost premium is justified depends on the deployment context: for a high-volume system where synthesis quality is secondary, improved RAG is the better choice; for a system where answer completeness and source coverage are primary, the multi-agent's 15-chunk pooled context and higher answer relevancy justify the additional cost.

Three enterprise deployment scenarios favour the multi-agent architecture specifically. First, in corpora where source types have genuinely non-overlapping terminology, always querying all sources eliminates the risk that a relevant passage in one source type is never retrieved because the query vocabulary does not match that source's dominant register. Second, in high-stakes, low-volume deployments such as regulatory compliance review, legal research, and incident post-mortems, where answer completeness outweighs throughput and cost, the multi-agent's 0 % failure rate and comprehensive source coverage are non-negotiable properties. Third, in workflows where downstream automation consumes the `sources_used` field for audit or escalation routing, the multi-agent's guaranteed full-coverage trace has operational value independent of per-query answer quality.

7.3 Limitations

Proxy data sources. The corpus consists of GitHub Issues, Markdown documentation pages, and CNCF PDF whitepapers, which are open, public datasets chosen to simulate Jira, Confluence, and SharePoint. Real enterprise data differs in several ways that may affect generalisability: proprietary terminology reduces cross-source retrieval overlap; access control policies fragment documents across systems in ways not captured by a static corpus; document quality is more variable, including partially complete drafts, archived pages, and duplicates. Results on the proxy corpus may be more favourable to retrieval-based approaches than on real enterprise data, where noise and formatting inconsistency would test chunking and embedding robustness more severely.

Synthetic evaluation query set. The 90 evaluation queries were generated by prompting `gpt-oss-120b` from sampled corpus excerpts and manually reviewed for quality. Synthetic query sets are vulnerable to a systematic bias: the generating model may produce questions that match the phrasing and structure of corpus passages, making retrieval artificially precise. The manual review criterion, requiring that question wording must not quote or paraphrase the answer, partially mitigates this risk but cannot eliminate it. A query set generated by domain experts or drawn from real user logs would provide a stronger evaluation baseline.

Faithfulness measurement unavailable for agent-based systems. As documented in Table 6.2, the RAGAS faithfulness metric could not be computed for the single agent on any query type, or for the multi-agent on any type except unanswerable, because the LLM judge timed out on the systems' longer responses. Faithfulness, which measures the degree to which answer claims are directly supported by retrieved context, is the metric most directly associated with hallucination risk. Its absence prevents any conclusion about whether the agent-based systems produce more or less grounded content than the RAG variants. For the multi-agent in particular, which synthesises from 15 chunks across three sources, the risk of introducing claims that cannot be traced to a single retrieved passage is structurally higher than in single-context RAG generation.

Single-agent max-iterations failures. The 37-query failure rate (41 %) at `MAX_ITERATIONS`=15 distorts all single-agent quality metrics, since empty answers contribute zero to BERTScore and ROUGE-L. The multi-agent architecture addresses this failure mode structurally by eliminating the ReAct cycle, but the single-agent evaluation data remains in the thesis as the baseline against which the multi-agent improvement is measured. A production deployment of the single-agent would require a query classifier that routes clearly unanswerable queries to a simpler fallback, or a recovery mechanism that extracts a partial answer from intermediate trace steps before the iteration ceiling is reached.

BERTScore and ROUGE-L insensitivity to retrieval improvements. The lexical and semantic similarity metrics were insensitive to the HyDE and MMR improvements applied to the improved RAG pipeline: ROUGE-L factual changed from 0.346 to 0.349, and BERTScore F1 changed from 0.217 to 0.232, both within the noise floor of a 30-query subset. The answer relevancy metric, which measures semantic alignment with the question rather than lexical proximity to a reference answer, captured the

improvement (factual: 0.792 to 0.804; synthesis: 0.679 to 0.742). This suggests that reference-based metrics alone are insufficient to evaluate retrieval quality improvements in open-domain RAG settings where correct answers may be expressed through diverse paraphrases.

Multi-agent always-all-sources routing. The multi-agent's 100 % routing accuracy is a consequence of its architecture, as it always queries all three sources, rather than a reasoning capability. This design eliminates routing errors but introduces a different inefficiency: for queries clearly targeting a single source type, the two redundant source retrievals add latency and dilute context precision by including potentially irrelevant chunks. The evaluation corpus does not contain enough single-source queries with strong source-type signal to quantify this effect, but in a deployment where query intent is clearly source-specific, a hybrid approach, routing obvious single-source queries to the appropriate retriever while sending ambiguous queries to all sources, would be more efficient.

Single-evaluator rubric. The complexity and maintainability rubric scores in Section 6.3 were assigned by the author based on the implementation experience described in Chapter 5. Scores are justified by objective proxies, including lines of code, parameter count, and extensibility step count, but are not peer-reviewed or validated against a second independent implementation. The rubric means of 4.0, 3.2, and 2.2 for RAG, multi-agent, and single agent respectively should be treated as directional findings rather than precise measurements.

Single LLM evaluated. All four systems use `gpt-oss-120b` as the generating model, a reasoning model with internal chain-of-thought. The reasoning model's thoroughness, which leads it to issue up to nine tool calls per query, is the primary cause of the single agent's high token consumption and max-iterations failures. A non-reasoning instruction-tuned model might produce a very different latency and failure-rate profile for the agent-based systems. The improved RAG pipeline's HyDE call likewise uses `gpt-oss-120b` for hypothesis generation; a smaller or faster model for hypothesis generation might reduce the latency overhead of HyDE substantially. These results are therefore specific to `gpt-oss-120b` and may not generalise to other LLMs.

7.4 Connection to Prior Work

This evaluation contributes to the growing body of empirical comparisons between retrieval-augmented and agentic approaches to knowledge retrieval. Within the RAG taxonomy developed by Gao, Xiong, Gao, Jia, Pan, Bi, Dai, Sun and Wang (2023), the original RAG pipeline implemented here corresponds to the Advanced RAG paradigm: it applies embedding-based retrieval, cross-encoder reranking, and sliding-window chunking with overlap. The improved RAG pipeline extends this with two Advanced RAG techniques: HyDE, which Gao, Ma, Lin and Callan (2023) demonstrate improves zero-shot dense retrieval by aligning query vectors with the document embedding space, and MMR, which Carbonell and Goldstein (1998) propose as a diversity-based reranking criterion that reduces redundancy in retrieved sets while maintaining relevance. The confirmed 9 % synthesis answer relevancy improvement is consistent with the function of both techniques: HyDE improves recall on synthesis queries by generating hypotheses in document register, and MMR ensures the retrieved context covers multiple facets of the question rather than repeating the same aspect from different passages. Ferrazzi et al. (2026) independently confirm that HyDE "has emerged as one of the best performing" pre-retrieval techniques across their evaluation corpora, reinforcing the present finding.

The agent-based systems connect to a broader theoretical and empirical literature. The single-agent implementation follows the ReAct framework of Yao et al. (2023) closely, and its failure mode, in which context accumulation exhausts the token budget over long tool-call sequences, is consistent with the structural limitation that motivates multi-agent coordination: as Xi et al. (2023) observe, parallel sub-agent dispatch is the primary scalability mechanism for settings in which sub-goals are structurally independent. The multi-agent coordinator implemented in this thesis realises this design principle directly, dispatching sub-agents to each of the three semantically distinct source types and synthesising in a single bounded call. The 5.9$\times$ improvement in enterprise retrieval recall that (2025) attribute to overcoming the fixed-candidate-set constraint of static pipelines is directionally consistent with the multi-agent's answer relevancy lead over the RAG variants, though the present comparison measures answer relevancy rather than recall and the corpora differ.

The findings are most directly comparable to those of Ferrazzi et al. (2026), who evaluate agentic RAG against classical RAG across enterprise-relevant corpora and report that agentic systems provide no consistent quality improvement over well-tuned classical

RAG, while incurring substantially higher latency and cost. The single-agent comparison in this thesis reaches the same conclusion. However, the multi-agent experiment diverges from Ferrazzi et al.'s finding: the coordinator architecture achieves higher answer relevancy than both RAG variants on every applicable query type, demonstrating that the quality advantage depends critically on which agentic design is evaluated. The present thesis therefore extends Ferrazzi et al.'s analysis in two directions. First, the evaluation distinguishes between two structurally different agent designs, namely sequential ReAct and parallel coordination, rather than treating agentic RAG as a single category; this distinction explains why prior work finds no quality improvement while the present evaluation finds one for the multi-agent variant. Second, implementation complexity is treated as an explicit evaluation dimension with a five-criterion rubric and quantified scores, providing a structured complement to the quality and performance results that Ferrazzi et al. do not report.

The (2025) framework contextualises these findings within enterprise deployment requirements, noting that agentic systems require additional governance mechanisms, including source attribution, failure logging, and iteration monitoring, that the structured logging implemented in this thesis addresses through the `agent_trace` and `sources_used` fields. The four-system evaluation arc, progressing from original RAG through single-agent failure, multi-agent correction, and improved RAG, models the incremental diagnostic and improvement process that a real enterprise deployment team would undertake after observing poor performance in a first agentic deployment.

Taken together, these findings suggest that the cost of the single-agent ReAct architecture is not offset by quality gains in the setting evaluated here, but that the multi-agent coordinator and the improved RAG pipeline each represent a viable path to quality improvements, one through architectural restructuring and the other through retrieval enhancement. The conditions under which the multi-agent's answer quality advantage over improved RAG justifies its 2.5$\times$ cost premium, namely heterogeneous terminology, mandatory source coverage, and low-volume high-stakes queries, remain to be validated against real enterprise data.

8 Conclusion

8.1 Summary of Findings

This thesis compared classical and agent-based approaches to knowledge retrieval across a heterogeneous enterprise documentation corpus through four distinct system configurations, each building on the limitations identified in its predecessor. Both pipelines were implemented from scratch, shared the same embedding model, chunking strategy, reranker, and LLM, and were evaluated over 90 queries spanning four types, namely factual, synthesis, ambiguous, and unanswerable, against three simulated enterprise source types.

The initial two-system comparison between the original RAG pipeline and the single-agent ReAct system confirmed RAG's advantage on every measurable dimension. The RAG pipeline achieved a context precision of 0.546 on factual queries and 0.730 on synthesis queries, against the agent's 0.338 and 0.315. Answer relevancy followed the same pattern: RAG reached 0.792 on factual and 0.679 on synthesis queries; the agent reached 0.663 and 0.547 respectively, with 41 % of queries producing empty answers due to the ReAct iteration ceiling. The latency and cost differences were equally stark: the agent required 6.5× longer per query and consumed 11.2× more tokens. The agent's only structural advantage, namely auditable source routing, achieved a 73.3 % accuracy that did not translate into quality gains sufficient to offset these costs.

The multi-agent coordinator architecture, implemented as a direct response to the single agent's failure modes, eliminated every failure the single agent exhibited. By replacing the sequential ReAct cycle with a two-node LangGraph graph comprising a retrieval node that queries all three sources sequentially followed by a single synthesis call over a bounded 15-chunk context, the multi-agent achieved a 0 % failure rate and produced the highest answer relevancy of any system evaluated: 0.818 on factual queries and 0.770 on synthesis queries, compared to original RAG's 0.792 and 0.679 respectively.

Latency fell to 4.3 seconds mean and cost to $2.91 per 90 queries, representing 2.5× RAG's cost rather than 11.2×. Unanswerable faithfulness reached 0.839, the highest of any system, reflecting the bounded context's resistance to fabrication when no relevant material is retrieved.

The improved RAG pipeline, which applied HyDE and MMR to the original pipeline, demonstrated that significant quality gains are available within the classical architecture without structural change. Synthesis answer relevancy improved from 0.679 to 0.742, a gain of 9 %, at a cost increase of $0.02 per 90 queries. Latency increased from 2.5 to 4.3 seconds due to the extra hypothesis generation call. BERTScore and ROUGE-L were insensitive to these improvements, confirming that reference-based lexical metrics underestimate the practical benefit of advanced retrieval techniques in open-domain settings.

The complexity rubric assigns mean scores of 4.0 for both RAG variants, 3.2 for the multi-agent coordinator, and 2.2 for the single-agent system. The multi-agent is more complex to build and maintain than either RAG variant, but substantially more transparent and debuggable than the single-agent design it supersedes. Adding a new data source requires two steps for RAG and three for the multi-agent, compared to five for the single agent, where routing behaviour must be revalidated across all existing tools whenever the toolset changes.

8.2 Practical Recommendation

The four-system evaluation produces a differentiated practical recommendation that depends on the deployment context and its primary constraints.

The original RAG pipeline remains the appropriate starting point for most enterprise deployments. It is the fastest to implement, the cheapest to run ($1.18 per 90 queries, 2.5-second mean latency), and the simplest to debug and extend. For query mixes dominated by factual lookups against a coherent knowledge domain, no other configuration evaluated here offers a better cost-quality ratio.

The improved RAG pipeline (with HyDE and MMR) is the right incremental upgrade when synthesis quality is a stated requirement and the cost and infrastructure must remain unchanged. The 9 % synthesis answer relevancy gain at negligible additional cost, achieved without new services, dependencies, or architectural changes, makes

this the highest-return improvement available to a team operating an existing RAG deployment. A team already running the original RAG pipeline can apply HyDE and MMR by modifying three files (`config.py`, `retrieve.py`, `generate.py`), without touching the vector store, embedding model, or evaluation infrastructure.

The multi-agent coordinator is the preferred architecture when maximum answer quality across all query types is the priority and a 2.5× cost premium over RAG is acceptable. It delivers the best answer relevancy on both factual (0.818) and synthesis (0.770) queries, handles unanswerable queries with the highest faithfulness of any system (0.839), produces no empty-answer failures, and provides full source attribution via the structured `agent_trace` log. Its main operational constraints are the requirement for three maintained retrieval indexes and the inability to selectively target a subset of sources, as it always queries all three, which may introduce noise for queries clearly targeting a single domain.

The single-agent ReAct system as evaluated here should not be deployed for general knowledge retrieval. It offers no quality advantage over any alternative, its 41 % failure rate makes aggregate quality metrics unreliable, and its cost and latency are prohibitive for production use. The failure modes documented in this thesis, comprising context accumulation exceeding the iteration ceiling, reasoning tokens exhausting the answer budget, and the absence of an exit path for unanswerable queries, are not incidental to a particular implementation; they are structural consequences of the sequential ReAct cycle combined with a reasoning-heavy model. Any future deployment of a sequential agentic retrieval system must address all three failure modes explicitly before evaluation.

In practice, a graduated adoption path reflects the findings most directly: deploy the improved RAG pipeline as the production baseline, monitor synthesis failure cases through structured logging, and escalate to the multi-agent coordinator for query types or user segments where answer completeness consistently underperforms. This path minimises both initial implementation cost and operational risk while preserving the option to increase quality investment where it demonstrably matters.

8.3 Future Work

Three directions offer the highest return for extending this evaluation.

Real enterprise data validation. The proxy corpus approximates enterprise knowledge systems but cannot reproduce their full complexity: proprietary terminology, access-control fragmentation, inconsistent document quality, and mixed languages. Replicating this evaluation on a live enterprise corpus, with appropriate data governance controls, would test whether the quality and latency relationships observed here hold under real-world noise conditions. In particular, the multi-agent's always-all-sources design may behave differently when one source type contains terminology that interferes with synthesis queries targeting another; this effect is not observable in the thematically coherent cloud-native corpus used here.

Hybrid routing for the multi-agent architecture. The multi-agent's 100 % routing coverage is an architectural property rather than a capability, achieved by always querying all sources rather than by predicting which source is relevant. This introduces unnecessary latency and potential context dilution on queries with a clear single-source target. A lightweight query classifier trained to predict source relevance, routing clear single-source queries to one retriever while directing ambiguous or synthesis queries to all sources, would combine the multi-agent's reliability with the efficiency of targeted retrieval. The structured logs and routing ground-truth labels produced in this evaluation provide a natural training and evaluation set for such a classifier.

Cost and latency optimisation for HyDE. The improved RAG pipeline's latency increase from 2.5 to 4.3 seconds is attributable entirely to the HyDE hypothesis generation call, which uses the same `gpt-oss-120b` reasoning model as the answer generation step. Using a smaller, faster model for hypothesis generation, one capable of producing a fluent 2–3 sentence passage in a target domain without chain-of-thought reasoning, would reduce the latency overhead of HyDE substantially without degrading hypothesis quality. The evaluation framework developed here, which captures per-stage latency and token counts in structured logs, provides the instrumentation needed to measure this trade-off precisely without re-running the full 90-query evaluation after each model substitution.

Figures

3.1 Ingestion data flow: three source pipelines converging at shared JSONL files, then diverging into system-specific indexes. Chunking parameters (400-word sliding window, 60-word overlap) and the BGE-base-en-v1.5 embedding model are shared across both systems; the sole architectural difference is a single unified index for RAG versus three per-source indexes for the agent. 24

6.1 Per-criterion complexity scores from Table 6.7. Higher score indicates lower complexity or simpler maintainability. RAG leads on four of five criteria; the multi-agent occupies the intermediate position on all five; the single agent scores lowest on four of five. 68

Tables

2.1 Evaluation metrics employed in this thesis, organised by dimension. . . 19

3.1 Corpus statistics after acquisition and filtering. 22
3.2 Unified document schema shared by both retrieval systems. 23
3.3 Agent retrieval tools and their source-type scope. 30
3.4 Controlled and varied parameters across both systems. 33

4.1 Corpus statistics after acquisition, filtering, and normalisation. 37
4.2 Evaluation query taxonomy: four types mapping to Gao et al.'s RAG capability dimensions. 39
4.3 Query set schema for `data/eval/queries.jsonl`. 41
4.4 Complexity and maintainability rubric: five criteria scored 1–5 (5 = simpler / lower complexity). 45

5.1 Project dependencies with confirmed installed versions (verified 2026-08-17). 60
5.2 Log record schema. Shared fields are produced identically by both pipelines; agent-only fields have no equivalent in the RAG log. 61

6.1 Quality metrics by system and query type. BERTScore uses `microsoft/deberta-xlarge-mnli` with baseline rescaling; values above zero indicate performance above the baseline model. † Multi-agent factual context precision is unavailable: RAGAS timed out on that subset. 62
6.2 Faithfulness by system and query type. Unavailability is due to RAGAS LLM judge timeout on long responses. 64
6.3 Source routing accuracy by system and query type. Unanswerable queries are excluded because no source type is expected. Multi-agent routing is 100 % by architectural design. 65

6.4 Response latency (seconds) across all 90 queries per system. Failed queries (empty answers) are included in latency statistics — they consumed the full iteration budget before failing. 65
6.5 Mean latency (seconds) by query type and system. 66
6.6 Token consumption and estimated cost for 90 queries. Cost estimates use GPT-4o proxy pricing ($2.50/M input, $10.00/M output). † Single-agent input is understated; estimated true total is approximately 8.2M tokens ($21). 67
6.7 Complexity and maintainability rubric scores (1 = most complex, 5 = least complex). Improved RAG shares the RAG codebase with two additional configuration flags and receives identical scores. 71

7.1 System-level trade-off comparison across five evaluation dimensions. CP = context precision; AR = answer relevancy. 75

Listings

References

(2025), 'AgenticRAG: Agentic retrieval for enterprise knowledge bases', *arXiv preprint arXiv:2605.05538* .
URL: *https://arxiv.org/abs/2605.05538*

Asai, A., Wu, Z., Wang, Y., Sil, A. and Hajishirzi, H. (2023), 'SELF-RAG: Learning to retrieve, generate, and critique through self-reflection', *arXiv preprint arXiv:2310.11511* .
URL: *https://arxiv.org/abs/2310.11511*

Carbonell, J. and Goldstein, J. (1998), The use of MMR, diversity-based reranking for reordering documents and producing summaries, *in* 'Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval', ACM, Melbourne, Australia, pp. 335–336.

Es, S., James, J., Anke, L. E. and Schockaert, S. (2023), 'RAGAS: Automated evaluation of retrieval augmented generation', *arXiv preprint arXiv:2309.15217* .
URL: *https://arxiv.org/abs/2309.15217*

Ferrazzi, P., Cvjeticanin, M., Piraccini, A. and Giannuzzi, D. (2026), 'Is agentic RAG worth it? an experimental comparison of RAG approaches', *arXiv preprint arXiv:2601.07711* .
URL: *https://arxiv.org/abs/2601.07711*

Gao, L., Ma, X., Lin, J. and Callan, J. (2023), Precise zero-shot dense retrieval without relevance labels, *in* 'Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)', Association for Computational Linguistics, Toronto, Canada, pp. 1762–1777.
URL: *https://arxiv.org/abs/2212.10496*

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J. and Wang, H. (2023), 'Retrieval-augmented generation for large language models: A survey', *arXiv preprint arXiv:2312.10997* .
URL: *https://arxiv.org/abs/2312.10997*

Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J. and Neubig, G. (2023), 'Active retrieval augmented generation', *arXiv preprint arXiv:2305.06983* .
URL: *https://arxiv.org/abs/2305.06983*

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S. and Kiela, D. (2020), Retrieval-augmented generation for knowledge-intensive NLP tasks, *in* 'Advances in Neural Information Processing Systems', Vol. 33, pp. 9459–9474.
URL: *https://arxiv.org/abs/2005.11401*

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N. and Scialom, T. (2023), Toolformer: Language models can teach themselves to use tools, *in* 'Advances in Neural Information Processing Systems'.
URL: *https://arxiv.org/abs/2302.04761*

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K. and Yao, S. (2023), Reflexion: Language agents with verbal reinforcement learning, *in* 'Advances in Neural Information Processing Systems'.
URL: *https://arxiv.org/abs/2303.11366*

Sumers, T. R., Yao, S., Narasimhan, K. and Griffiths, T. L. (2023), 'Cognitive architectures for language agents', *Transactions on Machine Learning Research* .
URL: *https://arxiv.org/abs/2309.02427*

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z. and Wen, J.-R. (2023), 'A survey on large language model based autonomous agents', *arXiv preprint arXiv:2308.11432* .
URL: *https://arxiv.org/abs/2308.11432*

Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., Zheng, R., Fan, X., Wang, X., Xiong, L., Liu, Q., Zhou, Y., Wang, W., Jiang, C., Zou, Y., Liu, X., Yin, Z., Dou, S., Weng, R., Cheng, W., Zhang, Q., Qin, W., Zheng, Y., Qiu, X., Huang, X. and Gui, T. (2023), 'The rise and potential of large language

model based agents: A survey', *arXiv preprint arXiv:2309.07864* .
URL: *https://arxiv.org/abs/2309.07864*

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. and Cao, Y. (2023), ReAct: Synergizing reasoning and acting in language models, *in* 'International Conference on Learning Representations'.
URL: *https://arxiv.org/abs/2210.03629*

Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q. and Artzi, Y. (2020), BERTScore: Evaluating text generation with BERT, *in* 'International Conference on Learning Representations'.
URL: *https://arxiv.org/abs/1904.09675*